

Crivo

Manual técnico do projeto

Editor de pixel art baseado em matrizes, em JavaScript, com camadas e grupos, modos de mistura, animação com tags, seleção, pontilhado, histórico ilimitado, paletas clássicas, arquivos do Aseprite e exportação para jogos. Roda de três maneiras: app desktop para Windows, servidor local com API em Django e PostgreSQL, ou arquivo único no navegador. Inclui uma ferramenta de linha de comando em Java.

Copyright (©) 2026 Erik Pimenta

Versão 2.1.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 6

- Para que serve 6
- O que o Crivo faz 6
- Tecnologias 7
- Identidade visual 8
- Novidades da versão 2.1.0 8
- Novidades da versão 2.0.0 9
- Novidades da versão 1.2.0 9
- Novidades da versão 1.1.0 10

Como rodar 11

- As três maneiras 11
- Requisitos 11
- Maneira 1: app desktop (Crivo.exe) 12
- Maneira 2: servidor local e backend 13
- Maneira 3: arquivo único (Crivo.html) 14

Conhecendo a tela 15

- Menus 16

Ferramentas de desenho 18

- Modos de tinta 19
- Pixel perfect 20
- Simetria 20

Seleção, pontilhado e sombreamento 22

- Ferramentas de seleção 22
- Menu Seleção 23
- Copiar, recortar, colar e apagar 23
- Mover pixels 24
- Pontilhado 24
- Degradê pontilhado 25
- Clarear e escurecer 25

Visualização 27

Modo ladrilho 27

Cores e paletas 29

Cor principal e secundária 29

Paletas predefinidas 29

Suas paletas 29

Importar e exportar paletas 30

Gerador de tons 30

Escolher cor 30

Editor de paleta 30

Modos de cor 31

Ajustes de cor 31

Camadas e animação 33

Camadas 33

Grupos 34

Modos de mistura 34

Camadas de referência e de tiles 35

Quadros 36

Cels vinculados 36

Tags 36

Papel cebola 36

Slices 37

Tamanho da matriz, transformações e filtros 38

Recortar e aparar 38

Filtros na camada 39

Importação e exportação 40

Abrir arquivos e abas 40

Arquivos do Aseprite 40

Exportar imagem 41

GIF animado 41

PNG animado e quadros em .zip 42

Sprite sheet 42

Exportar dados da matriz 43
Importar imagens 45
O arquivo .crivo 45

Salvando o trabalho 47

Servidor ou navegador 47
Salvar, versões e conflitos 47
Rascunho automático 48

Por dentro: matemática e estruturas de dados 49

A matriz de pixels 49
Composição das camadas 49
Linhas: algoritmo de Bresenham 49
Pixel perfect: cantos em L 49
Elipses: ponto médio na caixa 50
Balde de tinta: preenchimento por varredura 50
Pontilhado ordenado: a matriz de Bayer 50
Degradê: projeção e distância 50
GIF: compressão LZW 51
RLE com varint 51
Histórico ilimitado 52
Cores em OKLab e OKLCH 52
Conversão para a paleta e sombreamento 52
Contorno por vizinhança 52
Escala por vizinho mais próximo 53
Deslocamento com repetição e aparo 53

Arquitetura do código 54

Módulos do editor 54
Ponto de extensão: features e app 55
Modos de mistura: a fórmula 56
Fluxo de eventos 56
Servidor Django 56
Código em inglês desde a 2.0.0 57

API 58

Autenticação 58
Rotas 58
Códigos de resposta 59
Exemplos 60
Ferramenta em Java 61
Configuração 62
Nomes antigos das variáveis 63
Antes de publicar na internet 63
Qualidade e segurança 64
Cabeçalhos de segurança 64
Proteções no servidor 64
Testes por conjunto 65
Análise estática 67
Integração contínua 67
Personalizando o Crivo 68
Uma paleta fixa nova 68
Uma ferramenta nova 68
Um recurso com menu próprio 68
Um modo de mistura novo 68
Um filtro novo 68
Um formato de exportação novo 69
Ícones, cores e atalhos 69
Atalhos do teclado 70
Solução de problemas 73

Capítulo 1

Visão geral

O Crivo é um editor de pixel art de código aberto que roda no navegador. Ele trata a imagem como ela realmente é: uma matriz de pixels, cada um com quatro canais (vermelho, verde, azul e transparência). Todas as ferramentas leem e escrevem direto nessa matriz, e a tela só mostra o resultado ampliado, sem suavização.

O projeto foi feito para ser baixado, usado, estudado e adaptado, e roda de três maneiras: como **app desktop** no Windows (`Crivo.exe`), com o **servidor local e o backend** (o Django guarda projetos e paletas num banco de dados, com histórico de versões e uma API para outros programas) ou como **arquivo único** (`Crivo.html`), aberto direto no navegador. O editor é o mesmo nas três; o capítulo 2 mostra como subir cada uma.

Para que serve

Para desenhar sprites, ícones, itens, personagens, cenários em blocos e animações curtas, e levar o resultado direto para um jogo. Além de PNG, GIF animado e sprite sheets, o Crivo exporta a própria matriz em JSON, CSV, JavaScript, Python, C e Java, para quem escreve o jogo do zero e quer os pixels como dados.

Também serve como material de estudo de computação gráfica de baixo nível: o código implementa, sem bibliotecas, a manipulação de `ImageData`, o algoritmo de linha de Bresenham, a elipse de ponto médio, o preenchimento por varredura, a composição alfa, a codificação RLE, a compressão LZW do GIF, o pontilhado ordenado de Bayer e a conversão de cores para o espaço OKLab.

O que o Crivo faz

Desenho

- Lápis e borracha com pincel de 1 a 32 pixels, quadrado ou redondo, e a opção **pixel perfect** no lápis, que tira os cantos em L dos traços de 1 pixel; spray, curva, contorno livre, polígono, desfocar e embaralhar; **modos de tinta** e **pincel personalizado**.
- Balde de tinta com 4 ou 8 vizinhos, tolerância de cor, modo contíguo ou global e leitura da camada atual ou de todas.
- Linha, retângulo e elipse, vazados ou preenchidos, com Shift para 45°, quadrado e círculo perfeitos, e Alt para desenhar a partir do centro.
- **Pontilhado** (dithering) com o padrão de Bayer 4×4 e **degradê pontilhado** linear ou radial entre as cores principal e secundária.
- **Clarear e escurecer** pela rampa da paleta ou pela luminosidade OKLCH, para sombrear sem trocar de cor na mão.
- **Seleção** retangular, elíptica, laço, laço poligonal e varinha mágica, com somar, subtrair, interseção, inverter, expandir, contrair e transformar; copiar, recortar, colar, apagar e **mover pixels**; com uma seleção ativa, todas as ferramentas de pintura ficam presas dentro dela.
- Conta-gotas, e Alt + clique para pegar uma cor com qualquer ferramenta de desenho.
- Simetria horizontal, vertical ou nos dois eixos, valendo para todas as ferramentas de pintura.

Organização

- Camadas com visibilidade, opacidade, bloqueio, duplicação, reordenação e mescla, **grupos**, **19 modos de mistura**, camadas de referência e camadas de tiles.

- Quadros de animação com duração própria, duplicação, reordenação, FPS, **cels vinculados**, **tags**, slices, papel cebola configurável e reprodução, e vários projetos abertos em **abas**.
- Histórico de desfazer e refazer sem limite de passos, guardando só o que mudou.
- Matriz de 1×1 até 1024×1024, com âncora para centralizar ou expandir, escala por vizinho mais próximo, giro de 90°, espelhamento, recorte para a seleção e aparo das bordas transparentes.
- Filtros na camada: contorno de 1 pixel, conversão para a paleta atual, substituição de cor em todos os quadros, inversão e tons de cinza, espelhamento da seleção e deslocamento de meia matriz para emendar ladrilhos.
- Modo ladrilho, que repete o quadro em volta da matriz, e grade forte configurável (4, 8, 16 ou 32 pixels).

Cores, importação e exportação

- Paletas Crivo, Game Boy, NES, Pico-8, CGA, CGA modo 4, Commodore 64 e 1 bit, além das suas, que você cria, importa e exporta.
- Gerador de tons em OKLCH com desvio de matiz, editor de paleta, modos de cor RGBA, indexado e tons de cinza e ajustes de cor com prévia.
- PNG e JPEG em tamanho real ou ampliados, GIF e PNG animados com todos os quadros e transparência, quadros em PNG num `.zip`, arquivos `.aseprite` e sprite sheet horizontal, vertical ou em grade, com JSON das posições no formato do Crivo ou do Aseprite.
- Dados da matriz em JSON, CSV, JavaScript, Python, C e Java, no editor e também pela API.
- Importação de imagens com detecção do tamanho do pixel e fatiamento de sprite sheets.

Tecnologias

Camada	Tecnologia	Papel
Editor	JavaScript (módulos ES), Canvas 2D, HTML5, CSS3	Toda a interface e o núcleo de desenho, sem bibliotecas externas
Testes do editor	Node.js 20+ (`node:test`)	107 testes do núcleo: cores, geometria, balde, RLE, documento, histórico, formatos, exportação, seleção, filtros, pontilhado, GIF, modos de mistura, grupos, tags, tilemaps, ajustes, `.aseprite`, APNG e ZIP
App desktop	Go 1.24, Microsoft Edge WebView2	Janela própria do Windows, maximizada, com o editor do `Crivo.html` embutido no `Crivo.exe`
Arquivo único	Node.js (gerador sem bibliotecas)	Junta os módulos, o CSS e o ícone no `Crivo.html`, que abre sem servidor
Servidor	Python 3.13, Django 6.1, Gunicorn, WhiteNoise	API REST, validação, versões, paletas, PNG no servidor, cabeçalhos de segurança e entrega do editor
Banco	PostgreSQL 17 (ou SQLite)	Projetos, camadas, quadros, células, versões e paletas
Linha de comando	Java 21 (só o JDK)	Informações, paleta, PNG, sprite sheet e matriz em Java a partir de um `.crivo`
Qualidade	Ruff, Bandit, Playwright	Análise estática do Python, análise de segurança e teste ponta a ponta da interface no Chromium, com o servidor e no `Crivo.html`

Camada	Tecnologia	Papel
Execução	Docker Compose	PostgreSQL e aplicação com um comando
Integração contínua	GitHub Actions	Ruff, Bandit, testes em JavaScript, Django com PostgreSQL e Java, construção da imagem Docker e conferência do `Crivo.html` e do `Crivo.exe`

Identidade visual

O nome vem do crivo, a peneira de malha quadriculada: a mesma grade que organiza a pixel art. O logo é um ponteiro de mouse em pixels, em preto sobre branco, o primeiro desenho que quase todo mundo já viu feito pixel a pixel. É também o ícone do `Crivo.exe`.

A interface usa a paleta do projeto, também disponível como paleta de desenho:



Vinho	Ardósia	Café	Linho	Oliva	Rosé	Palha
#5D1C16	#A3B4BE	#5C4034	#E6DDCE	#7A6E46	#D6BDB8	#F6E4A6

No tema claro, o linho é o fundo, o vinho marca a ferramenta ativa e os botões principais, e a ardósia é a área em volta da matriz. No tema escuro, a palha assume o papel de destaque.

Novidades da versão 2.1.0

Área	O que entrou
Camadas	Grupos que abrem e fecham, 19 modos de mistura, camadas de referência (fora das exportações), camadas de tiles com tileset e propriedades da camada (`F2`) (capítulo 8)
Animação	Tags com direção e repetições, cels vinculados, duração por quadro, papel cebola configurável e slices (capítulo 8)
Ferramentas	Dez novas, 23 ao todo: spray, curva, contorno livre, polígono, desfocar, embaralhar, seleção elíptica, laço, laço poligonal e varinha mágica; modos de tinta e pincel personalizado (capítulos 4 e 5)
Seleção	Seleções de qualquer forma, modos somar, subtrair e interseção, menu Seleção com inverter, expandir, contrair, reselecionar e transformar com RotSprite (capítulo 5)
Cor	Modos de cor RGBA, indexado e tons de cinza, editor de paleta, seletor HSV e seis ajustes com prévia (capítulo 7)
Arquivos e tela	Abrir e exportar `.aseprite`, PNG animado (APNG), quadros em `.zip`, JSON do Aseprite, abas e janela de pré-visualização (capítulo 10)
Servidor, banco e Java	Campos novos na API, migração `0004_aseprite_features` (tabelas `tag`, `slice` e `tileset`), PNG do servidor com grupos e modos, comando `tags` na ferramenta em Java e um teste de paridade que exige os mesmos pixels no JavaScript, no Python e no Java (capítulos 13 a 17)

Área	O que entrou
Compatibilidade	Nada quebra: o `.crivo` continua na versão 2, só com chaves opcionais; arquivos da 2.0 abrem e salvam iguais; ao atualizar o servidor, rode `python manage.py migrate`

Novidades da versão 2.0.0

Área	O que mudou
Código em inglês	Identificadores, comentários, arquivos e pastas (o núcleo agora fica em `js/core`, a ferramenta em Java em `tools/java` e os exemplos em `examples`) passaram para o inglês. A interface, as mensagens e a documentação continuam em português, e o editor funciona do mesmo jeito
API	Mudança incompatível: as rotas agora ficam em `/api/v2`, com nomes, parâmetros e campos em inglês (capítulo 14)
Formato `.crivo`	Versão 2, com as chaves em inglês. Arquivos da 1.x continuam abrindo no editor, na API e na ferramenta em Java
Banco de dados	A migração `0003_english_schema` renomeia tabelas e colunas sem perder dados; versões antigas são convertidas ao restaurar
Configuração e comandos	Sete variáveis de ambiente com nomes novos (os antigos ainda funcionam), scripts `npm run serve`, `npm run portable` e `--check`, e comandos da ferramenta em Java em inglês, com os antigos como apelidos
Dados no navegador	Projetos, rascunho, paletas e preferências guardados pela 1.x são migrados sozinhos na primeira abertura

Novidades da versão 1.2.0

Área	O que entrou
Três maneiras de rodar	App desktop (`Crivo.exe`), servidor local com backend e arquivo único (`Crivo.html`), com o mesmo editor nas três
App desktop	`Crivo.exe` para Windows 10 e 11: abre maximizado, `F11` alterna a tela cheia e `Alt` + `F4` fecha gravando o rascunho; cria um atalho no menu Iniciar na primeira abertura; projetos guardados neste computador
Arquivo único	`Crivo.html` abre com dois cliques, sem servidor e sem backend, com Content-Security-Policy própria e sem nenhuma conexão de rede
Geradores	`npm run portable` e `npm run desktop`, sem bibliotecas no editor; o CI confere se os dois arquivos estão em dia com o código
Qualidade	21 testes ponta a ponta (10 com o servidor e 11 no `Crivo.html`, incluindo a integração com o app desktop) e build reproduzível do `Crivo.exe`

Novidades da versão 1.1.0

Área	O que entrou
Ferramentas	Seleção retangular (`S`), mover pixels (`V`), pontilhado (`D`), degradê pontilhado (`U`) e clarear e escurecer (`Q`)
Opções	Pixel perfect no lápis, pincel até 32 px, grade forte configurável e modo ladrilho (`T`)
Comandos	Selecionar tudo, desmarcar, copiar, recortar, colar, apagar a seleção, espelhar a seleção, recortar para a seleção, aparar bordas, deslocar meia matriz, contorno, converter para a paleta, substituir cor, inverter e tons de cinza
Exportação	GIF animado com todos os quadros, transparência, atraso de cada quadro e repetição infinita, com codificador próprio (`Ctrl` + `Shift` + `G`)
API	PNG gerado pelo servidor (`/png`) e exportação em C e Java
Segurança	Content-Security-Policy e outros cabeçalhos, HSTS, limite de escritas por IP, recusa de chave secreta fraca em produção, proteção contra JSON aninhado e miniaturas falsas
Qualidade	40 testes do núcleo, 39 testes da API (com um conjunto só de segurança), teste ponta a ponta com Playwright, Ruff e Bandit no CI

Capítulo 2

Como rodar

O Crivo sobe de três maneiras. O editor é o mesmo em todas; o que muda é o que precisa estar instalado e onde os projetos ficam guardados. Os arquivos `.crivo` abrem nas três, então dá para levar um desenho de uma para outra.

As três maneiras

Maneira	Como subir	Precisa de	Onde ficam os projetos
1. App desktop `Crivo.exe`	Dois cliques no `Crivo.exe` (depois, pesquise **Crivo** no menu Iniciar); abre maximizado	Windows 10 ou 11 (64 bits); nada para instalar	Neste computador, em `%LOCALAPPDATA%\Crivo`
2. Servidor local e backend Django + banco	`start.cmd`, `./start.sh` ou `python manage.py runserver`	Docker Desktop, ou Python 3.12 ou mais novo	No banco de dados (PostgreSQL ou SQLite), com versões e API
3. Arquivo único `Crivo.html`	Dois cliques no `Crivo.html`; abre no navegador	Um navegador atual; nada para instalar	No navegador em que foi aberto (IndexedDB)

Para só desenhar no Windows, o app desktop é o caminho mais direto. Para usar a API, o histórico de versões ou guardar os projetos num banco, suba o servidor. Para abrir em qualquer sistema sem instalar nada, use o `Crivo.html`.

Requisitos

Jeito de rodar	Precisa de	O que você ganha
App desktop	Windows 10 ou 11 (64 bits), com o Microsoft Edge WebView2, que já vem no sistema	O editor numa janela própria, maximizada, sem instalar nada
Docker (recomendado para o servidor)	Docker Desktop ou Docker Engine com Compose	Editor, API e PostgreSQL prontos, com versões dos projetos
Python	Python 3.12 ou mais novo	Editor e API com SQLite, sem instalar banco
Arquivo único	Um navegador atual	O editor completo, salvando no navegador
Só o editor num servidor de arquivos	Node.js 20 ou mais novo (ou qualquer servidor de arquivos)	O editor completo, salvando no navegador
Linha de comando	JDK 21 ou mais novo	Converter arquivos `.crivo` em PNG, sprite sheet e código Java
Gerar o `Crivo.exe`	Node.js 20 e Go 1.24 (qualquer sistema)	Recriar o app desktop a partir do código

Maneira 1: app desktop (Crivo.exe)

Baixe o `Crivo.exe` da raiz do repositório e dê dois cliques. O Crivo abre **maximizado** (janela cheia, com a barra de título e a barra de tarefas à vista), sem servidor, sem backend e sem navegador. O arquivo pode ficar em qualquer pasta, inclusive na Área de Trabalho; não há instalador.

1. Dê dois cliques no `Crivo.exe`.
2. Se aparecer **O Windows protegeu o computador**, clique em **Mais informações** e depois em **Executar assim mesmo**. O aviso aparece porque o executável não tem uma assinatura digital paga; ele não pede permissão de administrador.
3. O editor abre maximizado, e a barra de status mostra **Desktop (este computador)**.
4. Na primeira abertura, o Crivo cria um atalho no **menu Iniciar**, só para o seu usuário e sem pedir administrador. Depois disso, basta pesquisar **Crivo** no Windows.

Tela cheia e atalhos do app

Tecla ou menu	O que faz
`F11`	Leva à tela cheia, sem barra de título nem barra de tarefas, e volta para a janela maximizada, que pode ser movida, redimensionada e minimizada
`Alt` + `F4`	Fecha o Crivo. Antes de fechar, o app grava o rascunho, então nada do que estava na tela se perde
Exibir → Tela cheia	O mesmo que o `F11`, pelo menu
Arquivo → Sair do Crivo	O mesmo que o `Alt` + `F4`, pelo menu

Os dois itens de menu e o `F11` só aparecem no app desktop. Todos os outros atalhos do capítulo 19 valem igual.

Onde ficam os projetos

Projetos, paletas e o rascunho automático ficam em `%LOCALAPPDATA%\Crivo` (por exemplo, `C:\Users\voce\AppData\Local\Crivo`) e continuam lá depois de fechar o Crivo ou trocar o `Crivo.exe` por uma versão nova. Para levar um desenho para outro computador, use **Arquivo → Baixar arquivo .crivo**: o arquivo vai para a pasta Downloads e abre em qualquer uma das três maneiras.

Como o app funciona

O `Crivo.exe` é um programa pequeno em Go (pasta `desktop/`, cerca de 3 MB) que abre uma janela do Windows com o Microsoft Edge WebView2, o mesmo motor do Edge, que já vem no Windows 10 e 11. Dentro dele vai o `Crivo.html` inteiro: ao abrir, o app grava a página em `%LOCALAPPDATA%\Crivo` e a carrega de lá, então o editor, os testes e a segurança são os mesmos do arquivo único.

- A janela abre maximizada; a tela cheia do `F11` tira a borda e a barra de título e cobre o monitor em que a janela está, como num navegador.
- O atalho do menu Iniciar aponta para o `Crivo.exe` que foi aberto. Se você apagar o atalho, ele não volta, a não ser que o `Crivo.exe` seja aberto de outra pasta.
- O app roda sem permissão de administrador, não abre portas de rede e desliga as ferramentas de desenvolvedor e o menu de contexto do navegador.

- Se o WebView2 estiver faltando (raro, num Windows 10 desatualizado), o Crivo avisa e oferece abrir a página de download da Microsoft.
- As propriedades do arquivo (botão direito → **Propriedades → Detalhes**) mostram a versão e o copyright.

Gerar o Crivo.exe a partir do código

```
npm run desktop
npm run desktop -- --check
```

O primeiro comando atualiza o `Crivo.html`, desenha o ícone a partir do logo do projeto (`docs/images/logo.svg`), grava a versão e o copyright nas propriedades do arquivo e compila o app para Windows de 64 bits, a partir de qualquer sistema. Precisa do Node.js 20 e do Go; a versão exata do Go fica em `desktop/go.mod`, e o Go baixa essa versão sozinho se faltar.

O build é reproduzível: o mesmo código gera sempre o mesmo `Crivo.exe`, byte a byte. O segundo comando gera o arquivo de novo numa pasta temporária e confere se é idêntico ao do repositório; o CI roda essa conferência a cada push.

Maneira 2: servidor local e backend

Sobe o editor junto com a API em Django e o banco de dados. É a maneira com histórico de versões dos projetos, API para outros programas e projetos guardados num banco, que vale para qualquer navegador que abrir o endereço.

Com Docker

1. Baixe o repositório (botão **Code → Download ZIP** no GitHub, ou `git clone`).
2. No Windows, dê dois cliques em `start.cmd`. No Linux ou macOS, rode `./start.sh` no terminal, dentro da pasta.
3. Na primeira vez, o script copia `.env.example` para `.env` e troca a senha do banco e a chave secreta do Django por valores aleatórios.
4. O Docker baixa a imagem do PostgreSQL, constrói a imagem do Crivo e sobe os dois serviços. As migrações rodam sozinhas e criam as paletas predefinidas.
5. Quando `/api/v2/health` responde, o navegador abre em http://localhost:8080.

```
./start.sh
docker compose logs -f web
docker compose down
docker compose down -v
```

O segundo comando acompanha o registro da aplicação, o terceiro desliga tudo mantendo os dados, e o último apaga também o volume do banco.

Sem Docker, com Python

```
python -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

No Windows, ative o ambiente com `.venv\Scripts\activate`. Sem nenhuma variável definida, o Django roda em modo de desenvolvimento e grava no arquivo `crivo.sqlite3`, na raiz do projeto. O editor abre em http://localhost:8000. Ao atualizar uma instalação da 1.x, rode o `migrate` de novo: a migração `0003_english_schema` renomeia as tabelas e colunas para o inglês sem perder dados (com Docker, isso acontece sozinho ao subir).

Para usar um PostgreSQL já instalado, defina `CRIVO_DB_ENGINE=postgresql` e as variáveis `CRIVO_DB_HOST`, `CRIVO_DB_PORT`, `CRIVO_DB_NAME`, `CRIVO_DB_USER` e `CRIVO_DB_PASSWORD` antes do `migrate`.

> Se você desligar o modo de desenvolvimento (`CRIVO_DEBUG=false`), defina também uma `CRIVO_SECRET_KEY` própria com pelo menos 32 caracteres: a partir da 1.1.0, o servidor se recusa a iniciar com a chave de desenvolvimento fora do modo de desenvolvimento.

Só o editor, num servidor de arquivos

```
npm run serve
```

O script `scripts/serve.js` é um servidor de arquivos estáticos sem dependências que entrega a pasta `editor/static/editor/` em http://localhost:5173. O editor percebe que não há API e passa a salvar projetos no IndexedDB e paletas no `localStorage` do navegador. A barra de status mostra **Modo local (navegador)**. Qualquer outro servidor estático funciona, inclusive GitHub Pages ou `python -m http.server` dentro de `editor/static/editor/`.

Maneira 3: arquivo único (Crivo.html)

Baixe o `Crivo.html` da raiz do repositório e dê dois cliques. O editor abre no navegador direto da pasta, sem servidor local, sem backend e sem internet, com todas as ferramentas, camadas, animação e exportações. A barra de status mostra **Modo local (navegador)**.

Os projetos e as paletas ficam guardados no navegador em que o arquivo foi aberto. Para guardar uma cópia ou levar um desenho para outro computador, use **Arquivo → Baixar arquivo .crivo** e, depois, **Arquivo → Abrir arquivo .crivo...** (`Ctrl` + `O`).

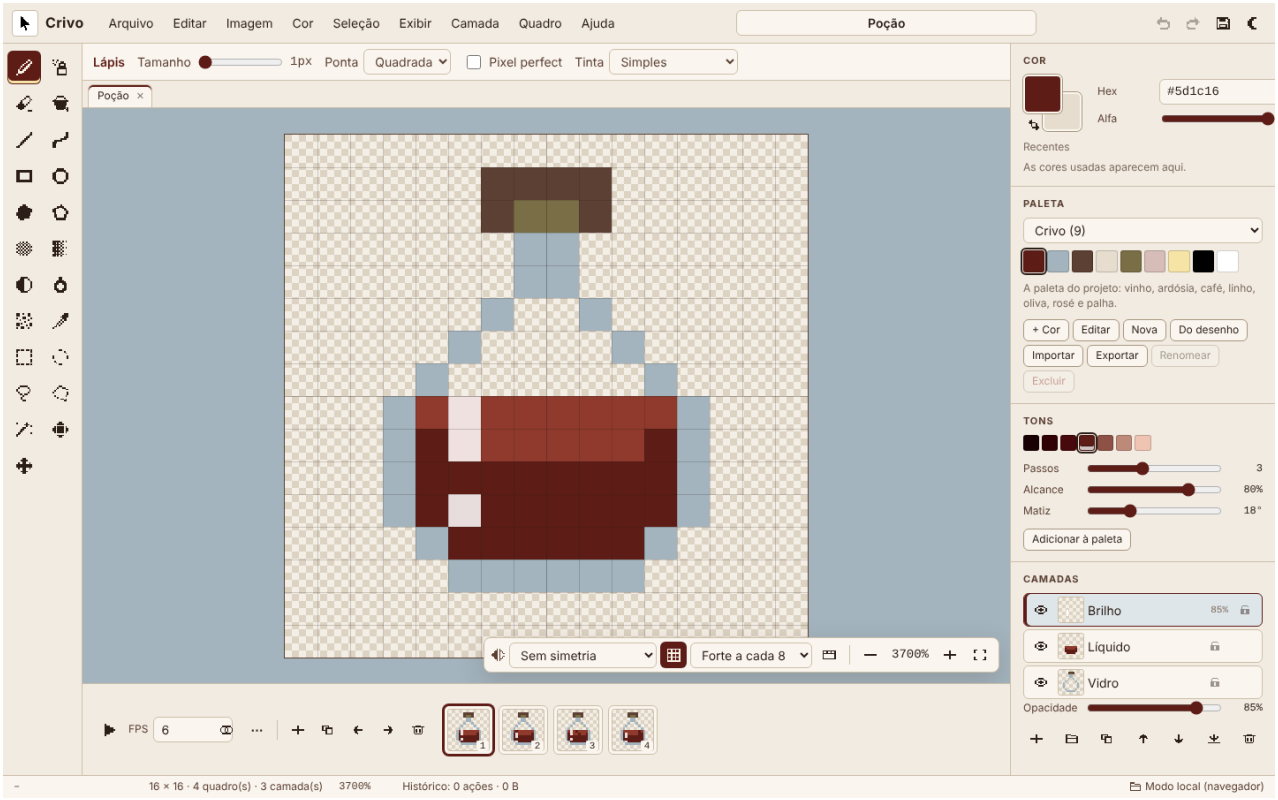
Por que um arquivo separado

O código do editor é escrito em módulos JavaScript, e os navegadores não carregam módulos de páginas abertas direto do disco (`file://`): por isso o `index.html` da pasta do editor não abre com dois cliques. O `Crivo.html` junta os 45 módulos num script só, embute o CSS e o ícone e traz a própria Content-Security-Policy numa tag `<meta>`: só executa o script embutido nele, conferido por hash SHA-256, não carrega nada de fora e não abre nenhuma conexão de rede.

Depois de mudar o editor, gere o arquivo de novo com `npm run portable` (o `npm run desktop` também faz isso); o CI confere se ele está em dia com `npm run portable -- --check`.

Capítulo 3

Conhecendo a tela



Tela principal com o exemplo da poção: ferramentas à esquerda, opções e abas no topo, matriz no centro, cores, paleta e camadas à direita e linha do tempo embaixo.

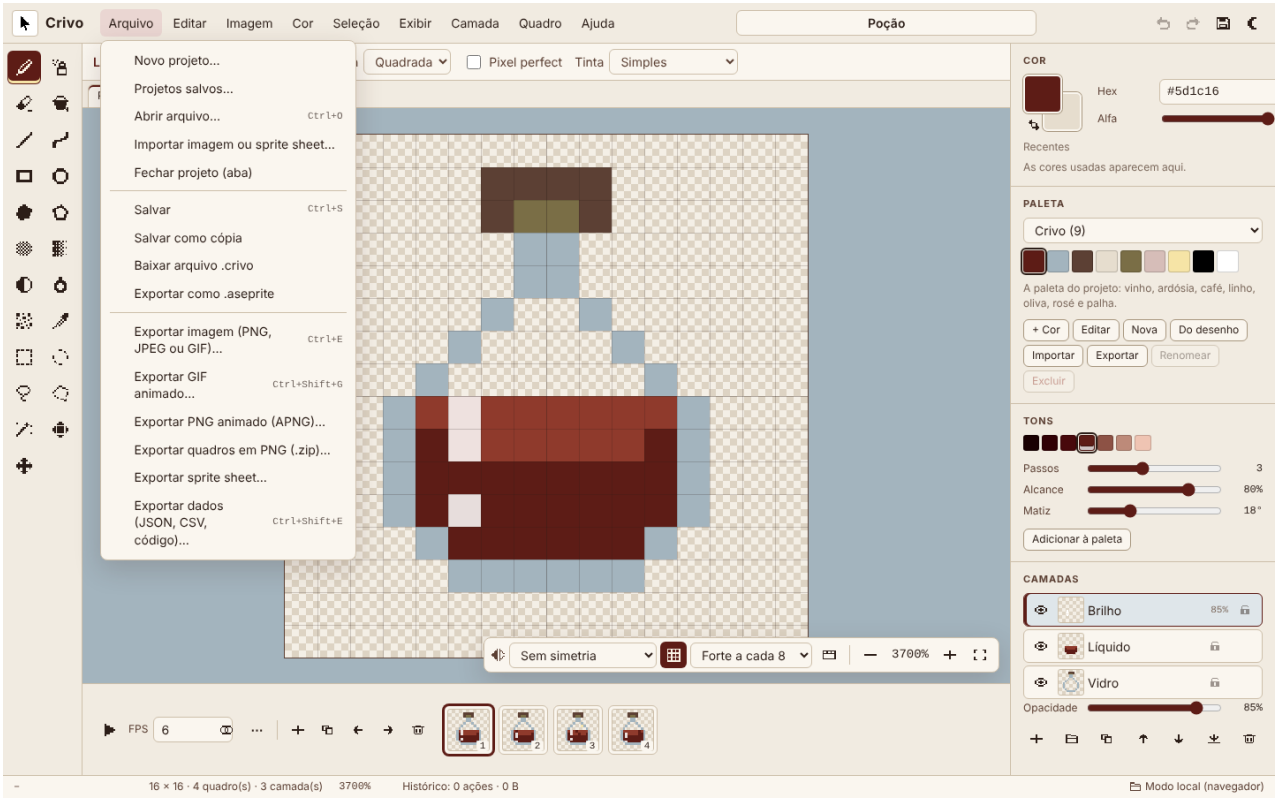
Área	O que tem
Topo	Logo, menus Arquivo, Editar, Imagem, Cor, Seleção, Exibir, Camada, Quadro e Ajuda, nome do projeto (clique para renomear), desfazer, refazer, salvar e tema
Ferramentas	As 23 ferramentas: lápis, spray, borracha, balde, linha, curva, retângulo, elipse, contorno livre, polígono, pontilhado, degradê pontilhado, clarear e escurecer, desfocar, embaralhar, conta-gotas, seleção retangular e elíptica, laço, laço poligonal, varinha mágica, mover pixels e mover a visão
Barra de opções	Opções da ferramenta ativa: tamanho e ponta do pincel, pixel perfect, modo de tinta, raio e densidade do spray, modo da borracha, vizinhos e tolerância do balde e da varinha, preenchimento das formas, modo de seleção, densidade do pontilhado, tipo e matriz do degradê, modo e intensidade do clarear e escurecer
Abas NOVO	Uma aba por projeto aberto, logo acima da matriz; clique do meio fecha
Visor	A matriz ampliada, com fundo quadriculado na transparência, grade, eixos de simetria, contorno tracejado da seleção (de qualquer forma), slices, grade de tiles, repetições do modo ladrilho e o contorno do pincel sob o cursor
Controles da visão	No canto do visor: simetria, grade, intervalo da grade forte, zoom e ajustar à tela
Painel lateral	Cor (principal, secundária, hexadecimal, alfa e recentes), paleta (com o botão Editar), gerador de tons e camadas (com grupos, modos de mistura e tiles)

Área	O que tem
Linha do tempo	Tocar, FPS, papel cebola e o botão ⋯ das configurações, ações dos quadros, as faixas das tags e as miniaturas de cada quadro, com a duração e o ∞ das células vinculadas
Pré-visualização NOVO	Janela flutuante (`F7`) com o quadro inteiro e o retângulo da parte visível
Barra de status	Posição do cursor, cor sob o cursor, tamanho da forma ou da seleção, tamanho do projeto, zoom, tamanho do histórico e onde os dados ficam salvos

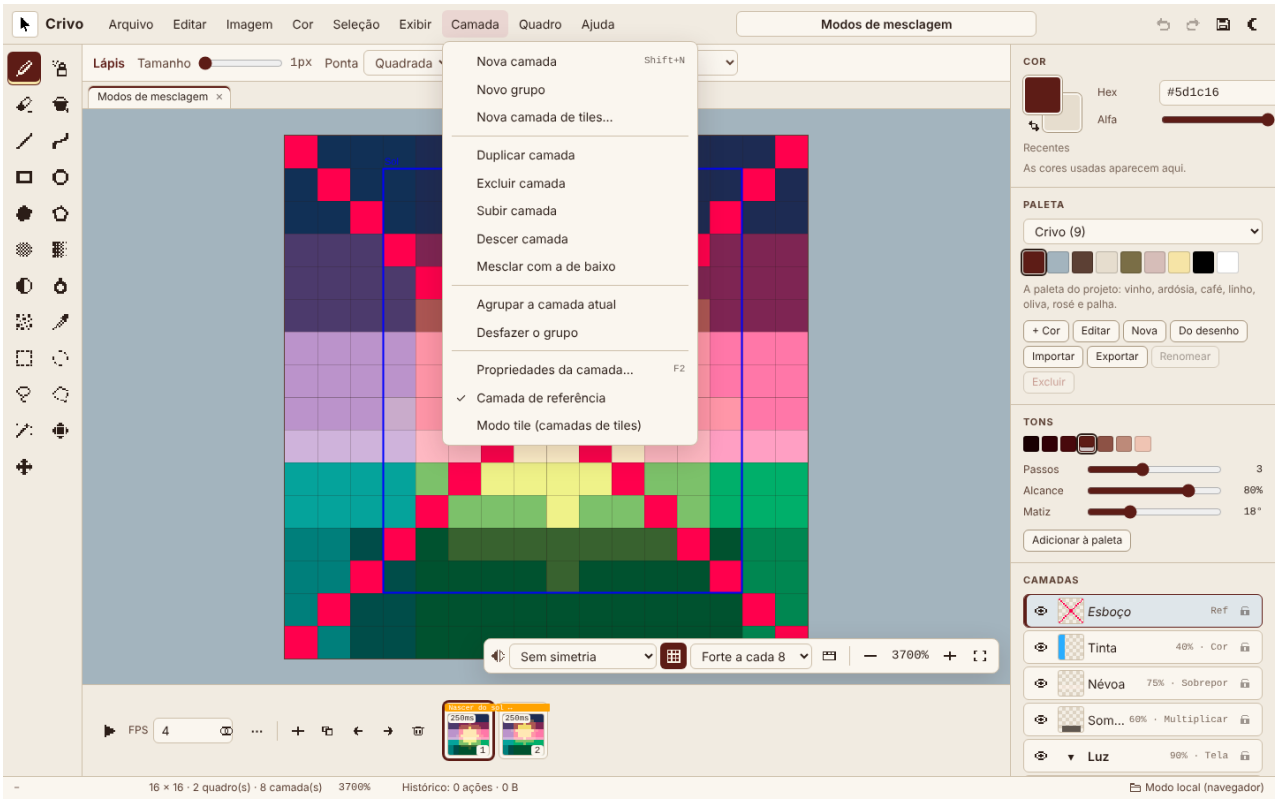
Menus

A 2.1.0 acrescentou quatro menus (Cor, Seleção, Camada e Quadro) e itens novos em Arquivo e Exibir, marcados em negrito.

Menu	Comandos
Arquivo	Novo projeto, projetos salvos, **abrir arquivo** (`.crivo`, `.json`, `.aseprite` e `.ase`, numa aba nova), importar imagem ou sprite sheet, **fechar projeto (aba)**, salvar, salvar como cópia, baixar `.crivo`, **exportar como .aseprite**, exportar imagem (PNG, JPEG ou GIF), GIF animado, **PNG animado (APNG)**, **quadros em PNG (.zip)**, sprite sheet e dados
Editar	Desfazer, refazer, copiar, recortar, colar, apagar, selecionar tudo, desmarcar, espelhar na horizontal e na vertical (seleção ou camada)
Imagem	Tamanho da matriz, recortar para a seleção, aparar bordas transparentes, girar 90° à direita e à esquerda, deslocar meia matriz (ladrilho), adicionar contorno (4 ou 8 vizinhos), converter para a paleta atual, substituir a cor primária pela secundária (todos os quadros), inverter cores e tons de cinza
Cor NOVO	Modo de cor RGBA, indexado ou tons de cinza, editar paleta, escolher cor principal e secundária, brilho e contraste, matiz/saturação/luminosidade, curva de cores, posterizar, remover ruído, matriz de convolução, converter para a paleta atual, substituir cor, inverter cores e tons de cinza
Seleção NOVO	Selecionar tudo, desmarcar, reselecionar, inverter seleção, expandir e contrair seleção, selecionar por cor, transformar seleção, usar seleção como pincel e pincel padrão
Exibir	Grade, papel cebola, **configurar papel cebola**, modo ladrilho, **slices**, **editar slices**, aproximar, afastar, ajustar à tela, tamanho real, tema claro ou escuro, **pré-visualização**, **próxima aba** e **aba anterior**
Camada NOVO	Nova camada, novo grupo, nova camada de tiles, duplicar, excluir, subir, descer, mesclar com a de baixo, agrupar a camada atual, desfazer o grupo, propriedades da camada, camada de referência e modo tile
Quadro NOVO	Novo quadro, novo quadro vinculado, duplicar, excluir, propriedades do quadro, vincular célula a quadros, desvincular célula, nova tag, editar e excluir a tag do quadro, tocar ou pausar a animação
Ajuda	Atalhos do teclado e sobre o Crivo



O menu Arquivo, com abrir arquivo, fechar aba, exportar como .aseprite, PNG animado e quadros em .zip.



O menu Camada com grupos, camada de tiles, propriedades e camada de referência.

Capítulo 4

Ferramentas de desenho

O botão esquerdo desenha com a cor principal e o direito com a cor secundária. Toda ferramenta de pintura respeita a simetria ativa e a seleção, e só altera a camada selecionada no quadro atual. Se ela estiver oculta ou bloqueada, o Crivo avisa e não desenha. A 2.1.0 trouxe dez ferramentas novas, marcadas abaixo; as de seleção estão no capítulo 5.

Ferramenta	Atalho	Como funciona	Opções
Lápis	B	Pinta pixel a pixel. Entre dois pontos do mouse, a linha de Bresenham preenche os buracos. Shift + clique traça uma reta desde o último ponto.	Tamanho (1 a 32), ponta quadrada ou redonda, pixel perfect, tinta, pincel
Spray NOVO	Shift + B	Espalha pixels ao acaso num círculo em volta do cursor enquanto o botão está pressionado.	Raio, densidade, tinta, pincel
Borracha	E	Igual ao lápis, mas deixa o pixel transparente ou pinta com a cor secundária.	Tamanho, ponta, apagar para transparente ou cor secundária
Balde de tinta	F	Preenche a área de cor parecida em volta do clique.	4 ou 8 vizinhos, tolerância (0 a 100%), só área contígua, ler da camada ou de todas
Linha	L	Arraste de uma ponta à outra. Shift trava em 0°, 45° e 90°.	Tamanho do pincel, tinta
Curva NOVO	Shift + L	Clique nos pontos de controle; a curva passa suave por eles. Duplo clique ou Enter aplica.	Tamanho do pincel, tinta
Retângulo	R	Arraste de um canto ao outro. Shift faz um quadrado, Alt desenha a partir do centro.	Preenchido, tinta
Elipse e círculo	O	Arraste a caixa da elipse. Shift faz um círculo, Alt desenha a partir do centro.	Preenchido, tinta
Contorno livre NOVO	Shift + D	Desenhe à mão livre; ao soltar, a forma fecha e é preenchida.	Tinta
Polígono NOVO	Shift + O	Clique nos cantos; duplo clique, Enter ou clique no primeiro ponto fecha.	Preenchido, tinta
Pontilhado	D	Pincel que aplica o padrão de Bayer 4×4: a cor principal onde o padrão manda, a secundária (ou nada) no resto.	Tamanho, ponta, densidade, fundo
Degradê pontilhado	U	Arraste do início ao fim: preenche a seleção, ou a camada inteira, com a transição em pontilhado ordenado.	Linear ou radial, Bayer 4×4 ou 8×8
Clarear e escurecer	Q	Botão esquerdo clareia e direito escurece os pixels sob o pincel, uma vez por traço.	Tamanho, ponta, rampa ou luminosidade, intensidade

Ferramenta	Atalho	Como funciona	Opções
Desfocar NOVO	Shift + Q	Troca cada pixel sob o pincel pela média dos vizinhos, para suavizar bordas.	Tamanho, ponta
Embaralhar NOVO	Shift + J	Troca pixels vizinhos de lugar sob o pincel, sem criar nem perder cores (textura de ruído).	Tamanho, ponta
Conta-gotas	I	Clique para pegar a cor do pixel; numa camada de tiles em modo tile, pega o tile.	Ler da camada ou de todas
Seleção retangular	S	Arraste para marcar um retângulo. Shift soma, Alt subtrai.	Modo de seleção
Seleção elíptica NOVO	Shift + S	Arraste a caixa de uma elipse.	Modo de seleção
Laço NOVO	A	Desenhe em volta da área à mão livre; ao soltar, a seleção fecha.	Modo de seleção
Laço poligonal NOVO	Shift + A	Clique nos cantos da área; fecha como o polígono.	Modo de seleção
Varinha mágica NOVO	W	Seleciona a área de cor parecida em volta do clique.	Modo, tolerância, contígua, vizinhos, ler de
Mover pixels	V	Arrasta o conteúdo da seleção (ou da camada inteira) no quadro atual. Setas movem 1 px; Shift + setas, 8 px.	—
Mover a visão	H	Arraste para mover a matriz pela tela.	—

No polígono, no laço poligonal e na curva, Backspace tira o último ponto e Esc cancela.

Modos de tinta NOVO

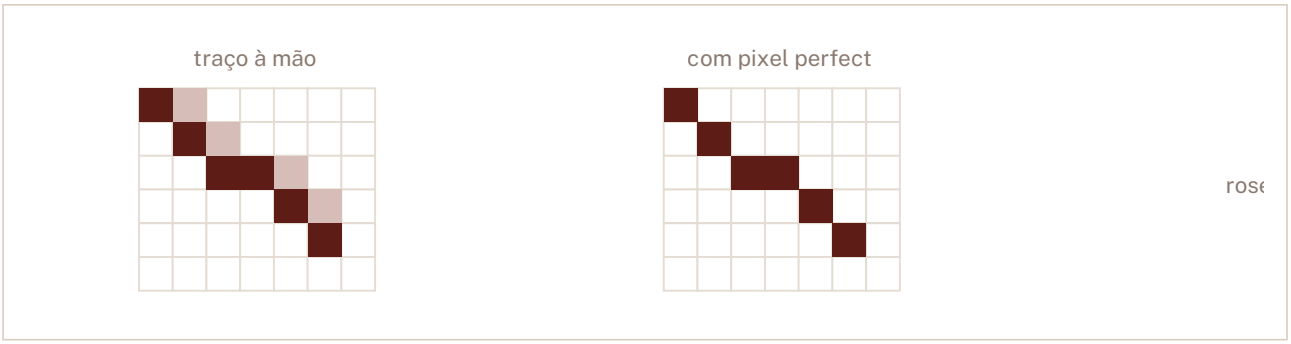
A opção **Tinta** da barra de opções vale para o lápis, a linha, o retângulo, a elipse, o spray, a curva, o contorno livre e o polígono:

Modo	O que faz
Simples	Troca o pixel pela cor, inclusive o alfa (o comportamento de sempre)
Composição alfa	Mistura a cor com o pixel de baixo, como uma tinta semitransparente sobre a camada
Travar alfa	Só pinta pixels que já têm alguma cor, mantendo o alfa deles: ótimo para recolorir sem sair do desenho
Atrás	Só pinta pixels transparentes, como se a tinta passasse por trás do que já está desenhado

> Durante o arraste das formas, da seleção e do degradê, a barra de status mostra a largura e a altura em pixels. Enquanto o botão do mouse está pressionado, os atalhos do teclado são ignorados, para que um toque acidental não troque de ferramenta ou desfaça algo no meio do gesto; só `Esc` funciona, e cancela a forma, o traço ou o arraste em andamento (na seleção, volta a seleção que havia antes).

Pixel perfect

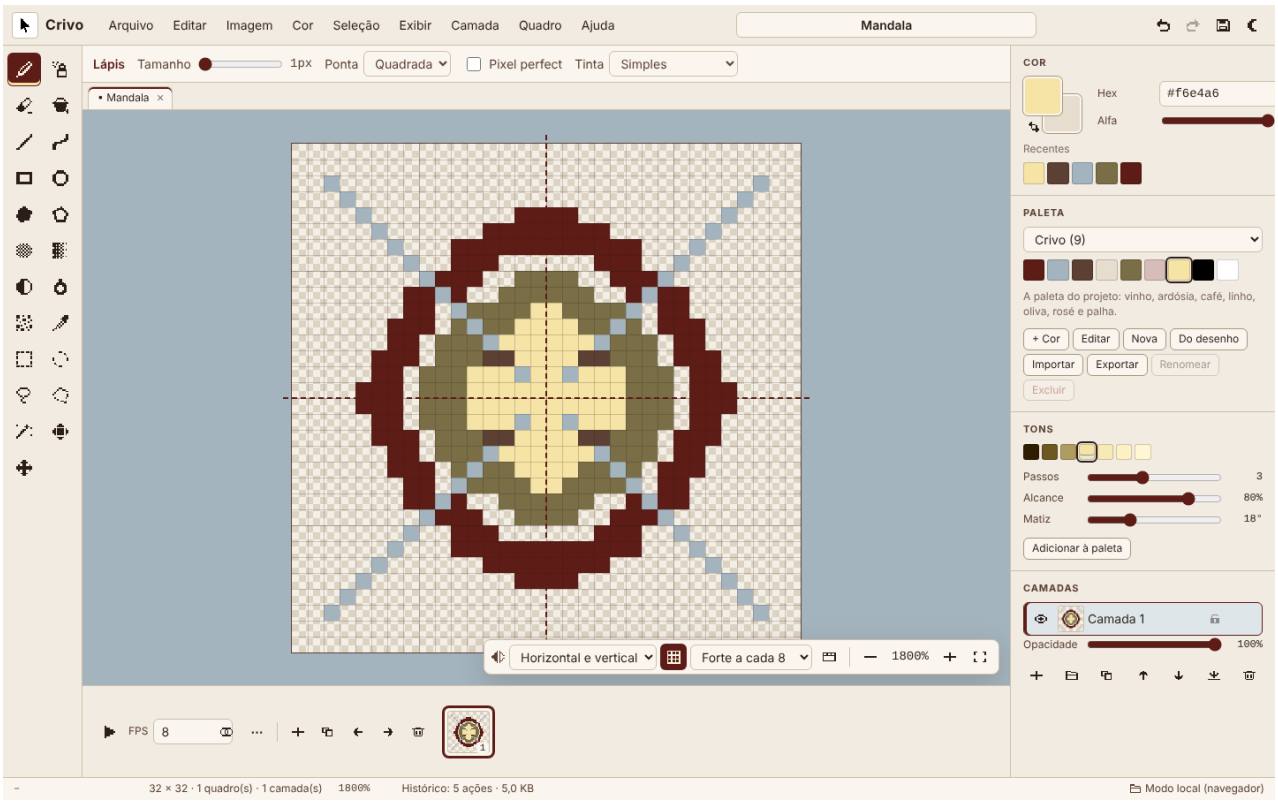
Num traço à mão livre de 1 pixel, o mouse quase nunca anda exatamente na diagonal: entre dois pixels diagonais aparece um terceiro, formando um "L" que deixa a linha grossa e serrilhada. Com a opção **Pixel perfect** ligada no lápis, o Crivo desfaz esses cantos enquanto você desenha, e a linha fica com exatamente um pixel de espessura. A opção vale para o pincel de 1 px; com pincéis maiores ela é ignorada.



Simetria

Os modos **espelho horizontal** (esquerda e direita), **espelho vertical** (cima e baixo) e **horizontal e vertical** (quatro quadrantes) espelham cada pixel pintado em torno do centro da matriz. Num desenho de largura L, o pixel da coluna x ganha um par na coluna L − 1 − x. Os eixos aparecem tracejados no visor e o contorno do pincel aparece também nas posições espelhadas.

A simetria vale para o lápis, a borracha, as formas, o balde (que preenche também a partir do ponto espelhado), o pontilhado e o clarear e escurecer. O atalho `M` alterna entre os modos.

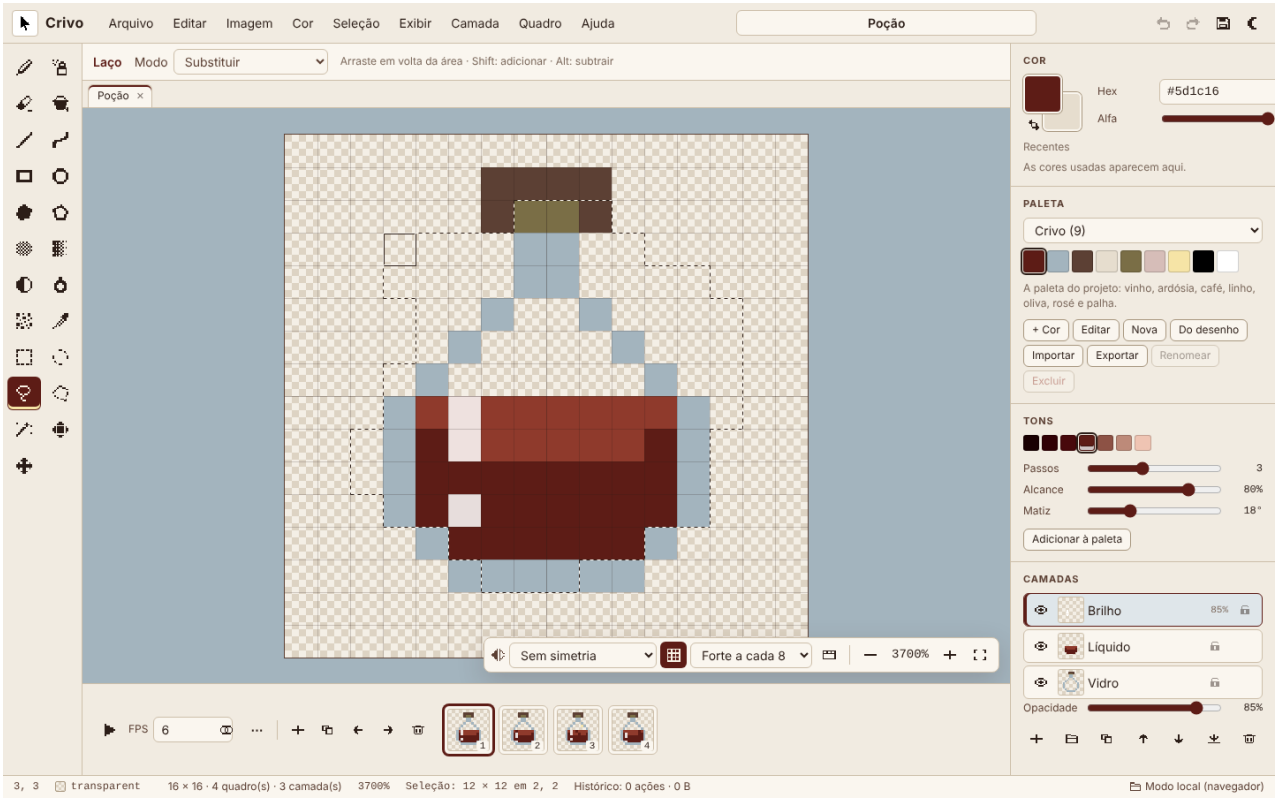


Simetria horizontal e vertical: os eixos tracejados dividem a matriz em quatro, e tudo o que é desenhado num quadrante aparece nos outros.

Capítulo 5

Seleção, pontilhado e sombreamento

Este capítulo cobre o que mais se usa no dia a dia da pixel art: trabalhar só numa parte do desenho, mover o que já foi desenhado, misturar duas cores sem criar cores novas e sombrear sem trocar de cor na mão. Na 2.1.0, a seleção deixou de ser só retangular.



Seleção feita com o laço sobre a poção; a barra de status mostra o retângulo que a contém.

Ferramentas de seleção

Há cinco: **seleção retangular** (`S`), **seleção elíptica** (`Shift` + `S`), **laço** (`A`), **laço poligonal** (`Shift` + `A`) e **varinha mágica** (`W`). O contorno tracejado (as "formigas") mostra a área, e a barra de status mostra a largura e a altura. Um clique sem arrastar, `Ctrl` + `D` ou **Seleção → Desmarcar** removem a seleção; `Ctrl` + `A` seleciona a matriz inteira.

Modo de seleção	Como usar
Substituir	O padrão: a área nova troca a seleção atual
Somar	Segure `Shift` (ou escolha na barra de opções) para juntar a área nova à seleção
Subtrair	Segure `Alt` para tirar a área nova da seleção
Interseção	Segure `Shift` + `Alt` para ficar só com o que está nas duas

Enquanto houver uma seleção, de qualquer forma, lápis, borracha, balde, formas, pontilhado, degradê, clarear e escurecer, filtros e ajustes só alteram pixels dentro dela, e copiar, recortar, colar, apagar, substituir cor, recortar para a seleção e mover pixels respeitam a forma exata.

Menu Seleção NOVO

Comando	O que faz
Reselecionar (`Ctrl` + `Shift` + `D`)	Volta a última seleção desfeita
Inverter seleção (`Ctrl` + `Shift` + `I`)	Seleciona tudo o que estava fora
Expandir e contrair seleção...	Cresce ou encolhe a borda de 1 a 10 pixels
Selecionar por cor (cor primária)	Seleciona todos os pixels da camada com a cor principal
Transformar seleção...	Gira em qualquer ângulo com o RotSprite (que preserva a pixel art sem criar cores novas), escala de 10% a 800% e espelha, com prévia; tudo vira um único passo do histórico
Usar seleção como pincel (`Ctrl` + `B`)	Transforma o conteúdo da seleção num pincel personalizado, usado pelo lápis, pelo spray e pela borracha; **Pincel padrão** volta ao pincel normal

Copiar, recortar, colar e apagar

Comando	Atalho	O que faz
Copiar	`Ctrl` + `C`	Guarda os pixels da seleção (ou da camada inteira) na área de transferência do Crivo
Recortar	`Ctrl` + `X`	Copia e deixa a área transparente

Comando	Atalho	O que faz
Colar	`Ctrl` + `V`	Cola na posição da seleção atual, ou no canto superior esquerdo, e já seleciona a área colada. A colagem fica "flutuando" sobre o fundo: com `V` ela se move sozinha, sem levar nem furar o que está embaixo, e os pixels transparentes dela deixam o fundo aparecer. Funciona entre camadas, quadros e projetos abertos na mesma aba
Apagar	`Delete`	Com seleção, apaga só ela; sem seleção, limpa a camada no quadro atual

A área de transferência fica na memória da aba: ela não vai para o sistema operacional nem é salva no projeto.

Mover pixels

A ferramenta **Mover pixels** (`V`) arrasta o conteúdo da seleção na camada e no quadro atuais; a seleção vai junto. Sem seleção, ela move a camada inteira. O lugar de onde os pixels saíram fica transparente. Com a ferramenta ativa, as setas do teclado movem 1 pixel e Shift + setas movem 8. Cada arrasto vira uma única entrada no histórico.

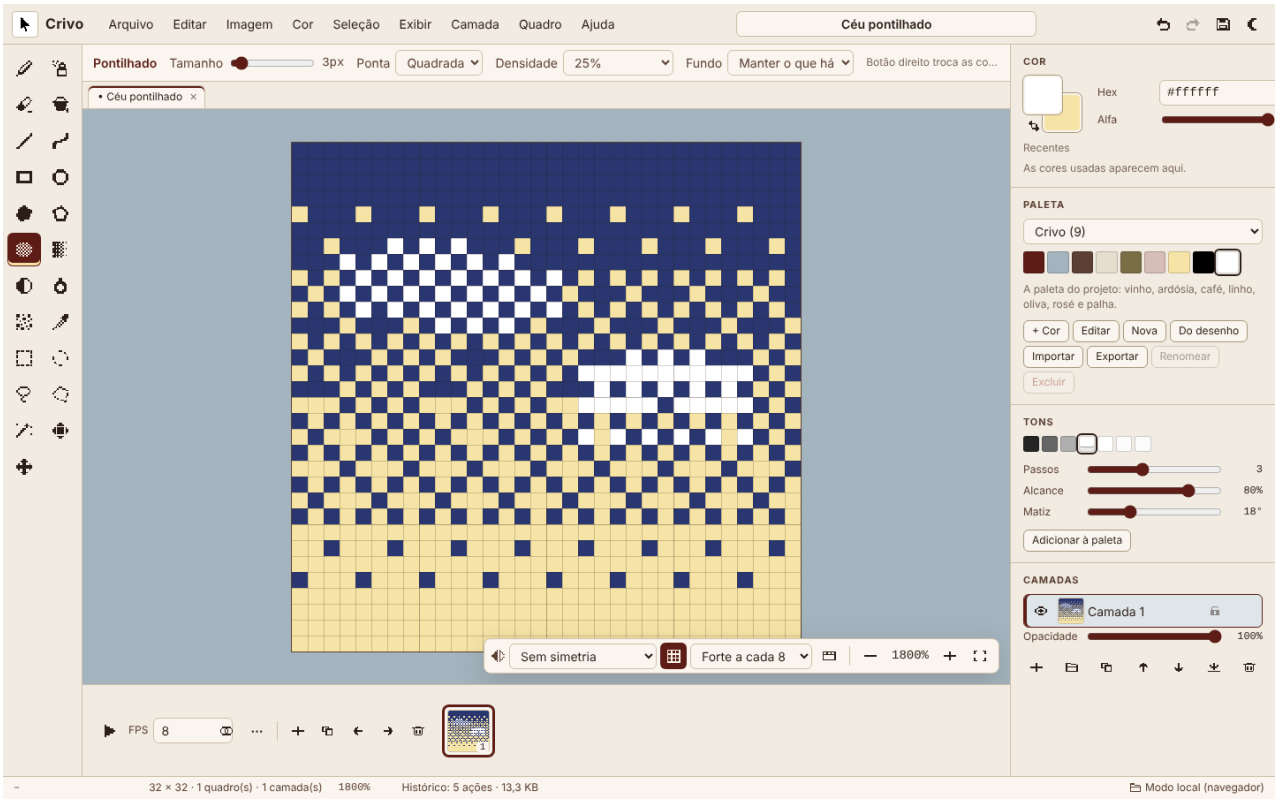
O conteúdo é "levantado" no primeiro movimento e continua guardado enquanto nada mais mudar no documento: arrastos e setas seguidos reaproveitam os mesmos pixels. Por isso dá para levar a seleção até fora da borda e trazer de volta sem perder nada; a seleção pode ficar em parte fora da matriz, e os comandos usam só a parte que cabe. Qualquer outra edição, desfazer, desmarcar ou trocar de camada "fixa" o conteúdo onde ele está. Uma colagem segue a mesma regra e, até ser fixada, se move sem arrastar o fundo.

Pontilhado

O **pontilhado** (`D`) é um pincel que, em vez de pintar todos os pixels sob ele, segue o padrão de Bayer 4×4: a cor principal vai para os pixels em que o limiar do padrão é menor que a densidade escolhida, e a secundária vai para os outros (ou eles ficam como estão, com **Fundo: manter**). O padrão é fixo na matriz, calculado pela posição (x, y) do pixel, então traços vizinhos se encaixam sem emendas e repassar o pincel não "suja" o desenho.

Densidade	Aparência
12,5%	Pontos isolados, para uma textura leve
25% e 37,5%	Um pixel em cada quatro (ou três em cada oito), em grade alternada
50%	Xadrez perfeito, a mistura clássica de duas cores
62,5%, 75% e 87,5%	O inverso de 37,5%, 25% e 12,5%

Degradê pontilhado



Céu feito com o degradê pontilhado e nuvens com o pincel de pontilhado.

O degradê pontilhado (U) preenche a seleção, ou a camada inteira se não houver seleção, com a transição da cor principal para a secundária. Arraste do ponto onde o degradê começa até onde ele termina; a prévia aparece durante o arraste (recalculada no máximo uma vez por quadro de vídeo, para não pesar em matrizes grandes) e o botão direito inverte as cores. Um clique sem arrastar não pinta nada e só mostra uma dica de como usar.

- **Linear**: a transição acompanha a direção do arraste; antes do início é só a cor principal, depois do fim é só a secundária.
- **Radial**: o início é o centro e a distância até o fim é o raio, para brilhos, sombras redondas e luzes.
- **Bayer 4×4** dá 17 níveis de mistura com um padrão mais marcado; **Bayer 8×8** dá 65 níveis e uma transição mais suave.

Como o resultado só tem as duas cores escolhidas, o degradê nunca cria cores fora da paleta, que é o que a pixel art pede.

Clarear e escurecer

A ferramenta **Clarear e escurecer** (Q) é um pincel de sombreamento: o botão esquerdo clareia e o direito escurece. Cada pixel muda uma única vez por traço, então passar o mouse de novo no mesmo lugar, sem soltar, não acumula. Pixels transparentes são ignorados.

Modo	Como funciona
Rampa da paleta	Ordena as cores da paleta atual pela luminosidade OKLab e troca cada pixel pela vizinha mais clara ou mais escura. Uma cor que não está na paleta vai primeiro para a mais próxima. O desenho continua só com cores da paleta
Luminosidade	Soma ou subtrai a intensidade (2% a 30%) no L do OKLCH, mantendo matiz e croma. Serve para ajustes finos em desenhos sem paleta fechada

Capítulo 6

Visualização

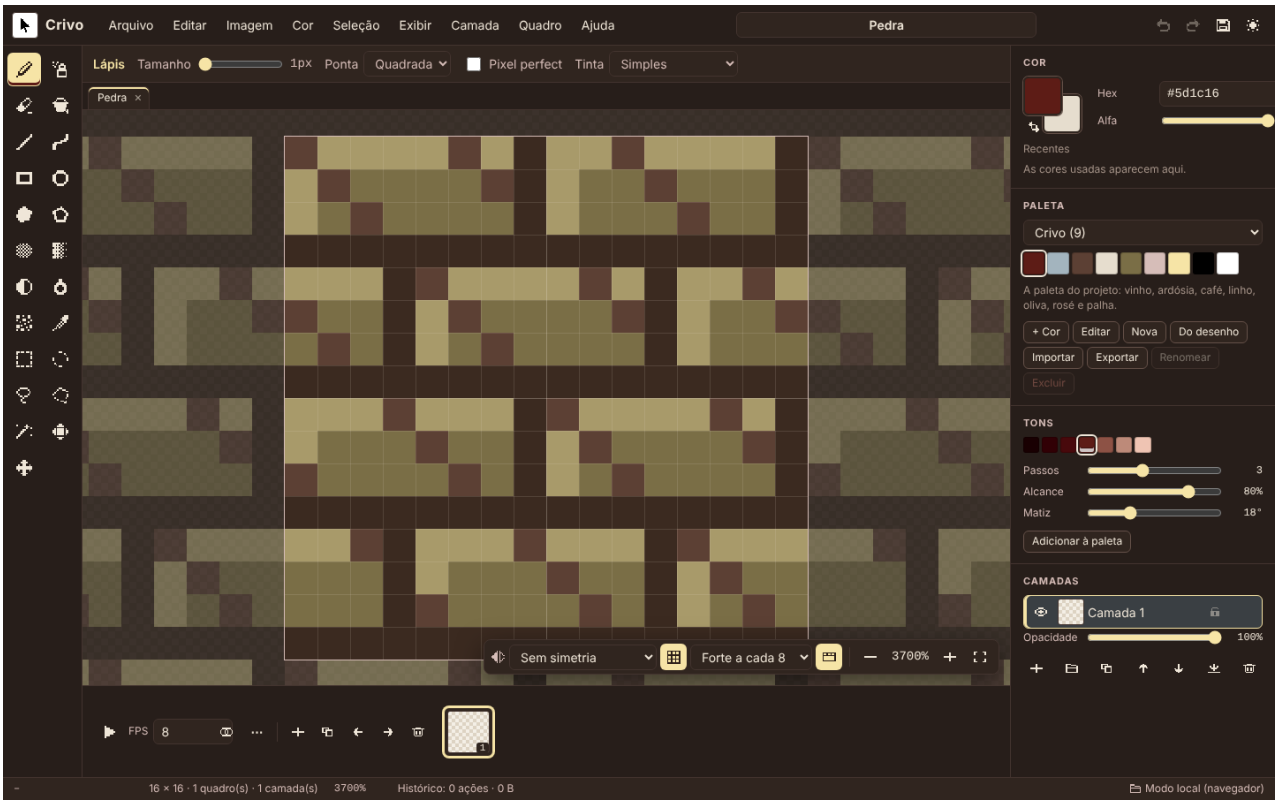
Ação	Como fazer
Aproximar e afastar	Roda do mouse (o ponto sob o cursor fica parado), botões − e +, ou as teclas `+` e `-`
Mover a visão	Segure `Espaço` e arraste, use o botão do meio do mouse, a ferramenta Mover, Shift + roda (horizontal) ou o arraste lateral do touchpad
Ajustar à tela	Tecla `0` ou o botão de ajuste: escolhe o maior zoom inteiro em que a matriz cabe inteira
Tamanho real	Tecla `1`: cada pixel da matriz vira um pixel da tela
Grade	Tecla `G`. Aparece a partir de 400% de zoom, com uma linha mais forte a cada 8 pixels (configurável)
Grade forte NOVO	No canto do visor, escolha a cada quantos pixels a linha forte aparece: 4, 8, 16, 32 ou desligada. Use o tamanho do seu tile (16 para blocos de 16×16, por exemplo)
Modo ladrilho NOVO	Tecla `T` ou **Exibir → Modo ladrilho**: repete o quadro atual 3×3 em volta da matriz, esmaecido

O zoom é sempre um número inteiro (de 100% a 12800%), para que cada pixel da matriz ocupe um quadrado exato de pixels da tela. O desenho é feito com `imageSmoothingEnabled = false`, que copia os pixels por vizinho mais próximo, sem borrar.

Modo ladrilho

Texturas de chão, paredes e fundos de jogo são repetidas lado a lado, e qualquer diferença entre a borda esquerda e a direita (ou a de cima e a de baixo) vira uma emenda visível. No modo ladrilho, o visor desenha oito cópias esmaecidas do quadro atual em volta da matriz, atualizadas a cada traço, para você ver a repetição enquanto desenha. As cópias são só visuais: você continua pintando na matriz do centro, e nada muda nas exportações.

Para pintar exatamente em cima das emendas, use **Imagem → Deslocar meia matriz** (`Shift` + `T`): a camada anda meia largura e meia altura com repetição, e as bordas vão para o meio da matriz. Depois de corrigir, desloque de novo para voltar à posição original.



Modo ladrilho ligado: o quadro se repete em volta da matriz para conferir as emendas.

Capítulo 7

Cores e paletas

Cor principal e secundária

O quadrado da frente é a cor principal (botão esquerdo) e o de trás, a secundária (botão direito). Clique num deles para abrir o seletor de cor do sistema. O campo **Hex** aceita `#5d1c16`, `5d1c16`, a forma curta `#a3b` ou oito dígitos com transparência, como `#5d1c1680`. O controle **Alfa** muda a opacidade da cor principal; pixels semitransparentes se misturam com o que estiver nas camadas de baixo.

A tecla `X` troca as duas cores. As 16 últimas cores usadas ficam em **Recentes**.

Paletas predefinidas

Paleta	Cores e origem
Crivo	As sete cores do projeto, mais preto e branco
Game Boy	Os verdes da tela do Game Boy original (DMG-01)
NES	As cores únicas da PPU 2C02, sem as repetições de preto
Pico-8	A paleta fixa do console de fantasia Pico-8
CGA	As 16 cores do adaptador CGA da IBM
CGA (modo 4)	Preto, ciano, magenta e branco, a combinação mais usada nos jogos CGA
Commodore 64	As cores do chip VIC-II
1 bit	Preto e branco

Suas paletas

- **Nova** cria uma paleta com as cores principal e secundária.
- **+ Cor** adiciona a cor principal. Numa paleta predefinida, o Crivo oferece criar uma cópia sua antes, porque as predefinidas não mudam.
- **Do desenho** junta todas as cores usadas em todos os quadros, da mais usada para a menos usada, até 256.
- `Ctrl` + clique numa cor remove a cor da paleta. **Renomear** e **Excluir** valem só para as suas paletas.
- Com o servidor, as paletas vão para o banco e aparecem em qualquer navegador. Sem servidor, ficam no navegador atual.
- A paleta atual também é usada pelo clarear e escurecer no modo rampa e por **Imagem → Converter para a paleta atual**.

Importar e exportar paletas

Formato	Extensão	Como é
Lospec / HEX	`.hex`	Uma cor por linha, como `5d1c16`
GIMP, Aseprite, Krita, Inkscape	`.gpl`	Cabeçalho `GIMP Palette`, nome e linhas `R G B nome`
Paint.NET	`.txt`	Uma cor por linha em `AARRGGBB`, com comentários iniciados por `;`
JSON	`.json`	`{"name": "...", "colors": ["#..."]}` ou só a lista de cores
Texto livre	`.txt`, `.csv`	Qualquer código hexadecimal encontrado no texto

Gerador de tons

O painel **Tons** cria uma rampa de sombras e luzes a partir da cor principal. Em vez de só escurecer e clarear em RGB, ele trabalha no espaço de cor OKLCH (luminosidade, croma e matiz), em que passos iguais de luminosidade parecem iguais aos olhos.

Controle	Efeito
Passos	Quantos tons de cada lado (1 a 6). A rampa tem 2 × passos + 1 cores
Alcance	Quanto as pontas da rampa se aproximam do preto e do branco
Matiz	Quantos graus as sombras giram em direção ao azul-violeta e as luzes em direção ao amarelo (o hue shifting da pixel art)

Clique num tom para usá-lo e em **Adicionar à paleta** para guardar a rampa inteira. Cores fora do que a tela consegue mostrar têm o croma reduzido por busca binária até caberem, sem mudar a luminosidade nem o matiz. Uma rampa adicionada à paleta é exatamente o que o clarear e escurecer no modo rampa percorre.

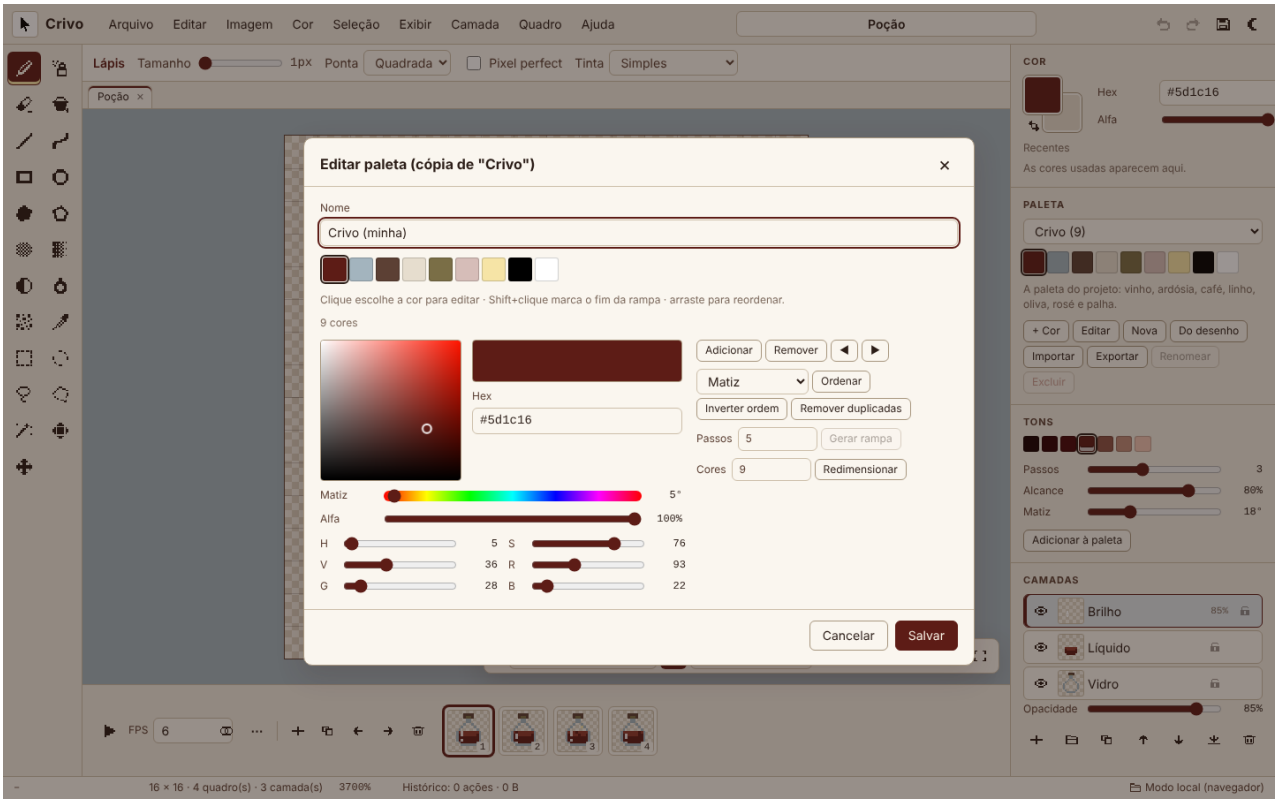
Escolher cor NOVO

Cor → Escolher cor principal... e **Escolher cor secundária...** abrem o seletor do Crivo: um quadrado de saturação e valor, as faixas de matiz e de alfa, os campos R, G, B e A e o hexadecimal, com a cor antiga ao lado da nova.

Editor de paleta NOVO

Cor → Editar paleta..., ou o botão **Editar** do painel, abre a paleta inteira para mudar de uma vez:

- clique numa cor para editá-la no seletor; **Adicionar**, **Remover** e as setas mudam a lista e a ordem;
- **Ordenar** por matiz, luminosidade ou saturação, **Inverter ordem** e **Remover duplicadas**;
- **Gerar rampa** cria os tons entre duas cores, e **Redimensionar** muda a quantidade de cores;
- uma paleta predefinida editada é salva como uma paleta sua; no modo indexado, mudar uma cor recolore todas as células que a usam, num único passo do histórico.



Editor de paleta com o seletor HSV, a ordenação e a rampa entre duas cores.

Modos de cor NOVO

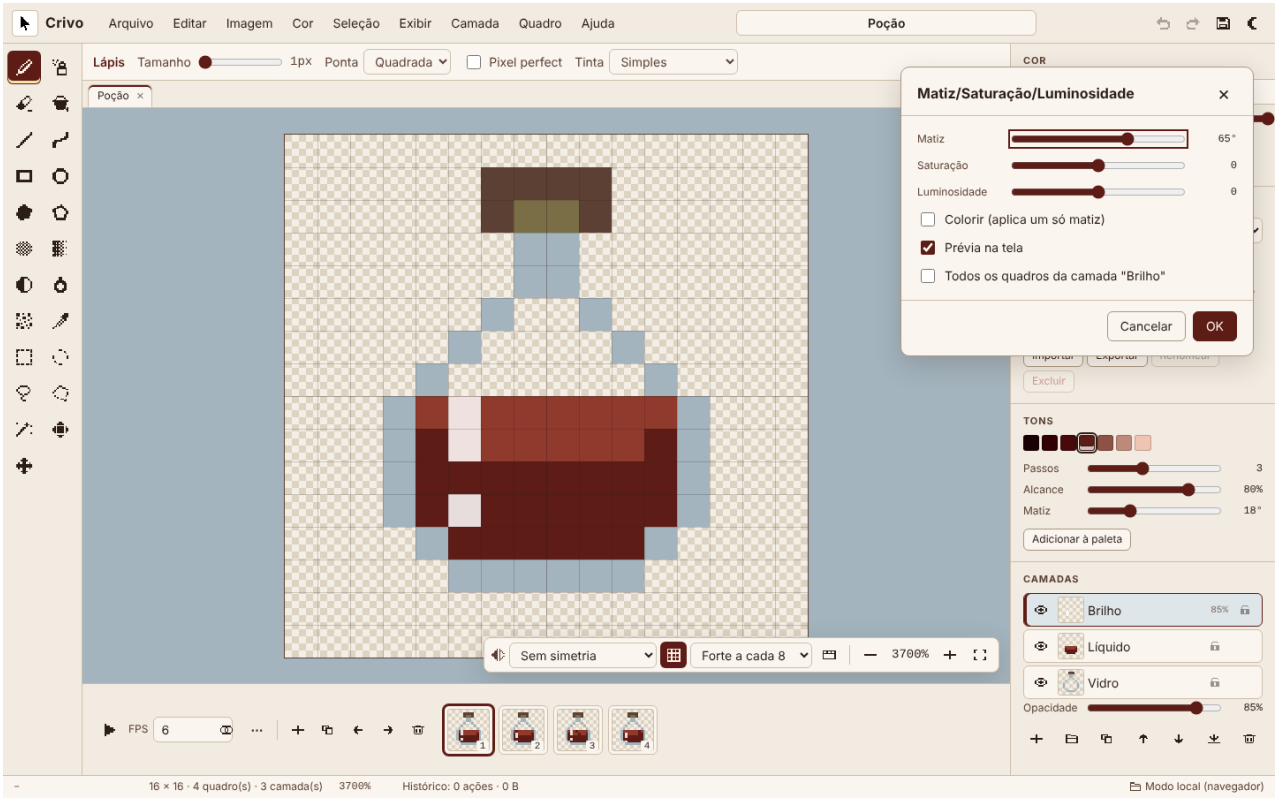
Modo	Como funciona
RGBA	O padrão: qualquer cor com transparência
Indexado	Toda cor vai para a mais próxima da paleta atual, também em traços, colagens e filtros. Ao converter, dá para usar o pontilhado Bayer 4×4 com a intensidade escolhida
Tons de cinza	Todas as cores viram cinza com a mesma luminosidade e continuam cinza ao desenhar

Ajustes de cor NOVO

Os ajustes do menu **Cor** mostram a prévia na tela enquanto você mexe; **Cancelar** volta ao que era e **OK** vira um único passo do histórico. Eles respeitam a seleção e podem valer para todos os quadros da camada.

Ajuste	O que faz
Brilho e contraste…	Clareia ou escurece e aumenta ou diminui a diferença entre os tons
Matiz/Saturação/Luminosidade… (`Ctrl` + `U`)	Gira o matiz, satura ou dessatura e clareia ou escurece; **Colorir** aplica um só matiz
Curva de cores… (`Ctrl` + `M`)	Uma curva por canal (ou para todos): clique adiciona um ponto, arrastar move e duplo clique remove
Posterizar…	Reduz o número de níveis de cada canal

Ajuste	O que faz
Remover ruído…	Troca pixels isolados pela cor que predomina em volta
Matriz de convolução…	Suavizar, nitidez, realce de bordas e desfoque gaussiano



Matiz, saturação e luminosidade com a prévia ligada.

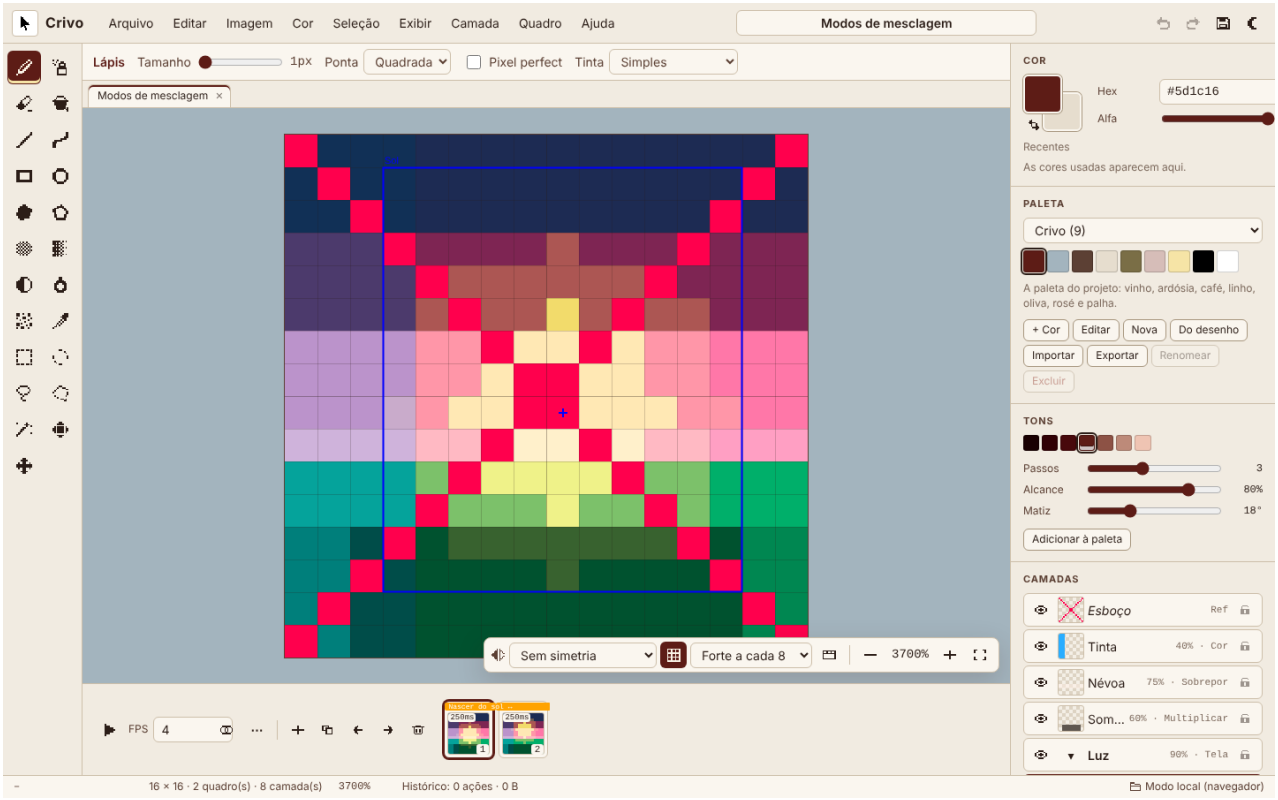
Capítulo 8

Camadas e animação

Camadas

A lista mostra a camada mais alta no topo. A camada selecionada recebe o desenho. Cada linha tem o olho (mostrar e ocultar), a miniatura, o nome, o cadeado e selos para a opacidade menor que 100%, o modo de mistura, **Ref** (referência) e **Tiles**. Os mesmos comandos estão no menu **Camada**.

Botão ou comando	O que faz
+ ou Nova camada (`Shift` + `N`)	Cria uma camada logo acima da selecionada
Novo grupo NOVO	Cria um grupo vazio; **Agrupar a camada atual** põe a camada selecionada num grupo novo e **Desfazer o grupo** devolve as camadas ao nível de cima
Duplicar	Copia a camada com o conteúdo de todos os quadros
Subir e descer	Muda a ordem de empilhamento
Mesclar	Junta a camada com a de baixo, aplicando a opacidade e o modo de mistura dela, e remove a de cima
Lixeira	Exclui a camada (o projeto mantém pelo menos uma)
Opacidade	De 0 a 100%. A mudança entra no histórico quando você solta o controle
Propriedades da camada... (`F2`) NOVO	Também com duplo clique na camada: nome, opacidade, modo de mistura e camada de referência



O exemplo blend-modes.crivo: o grupo Luz (tela, 90%) com Halo (adição) e Sol, e camadas em multiplicar, sobrepor e cor.

Grupos NOVO

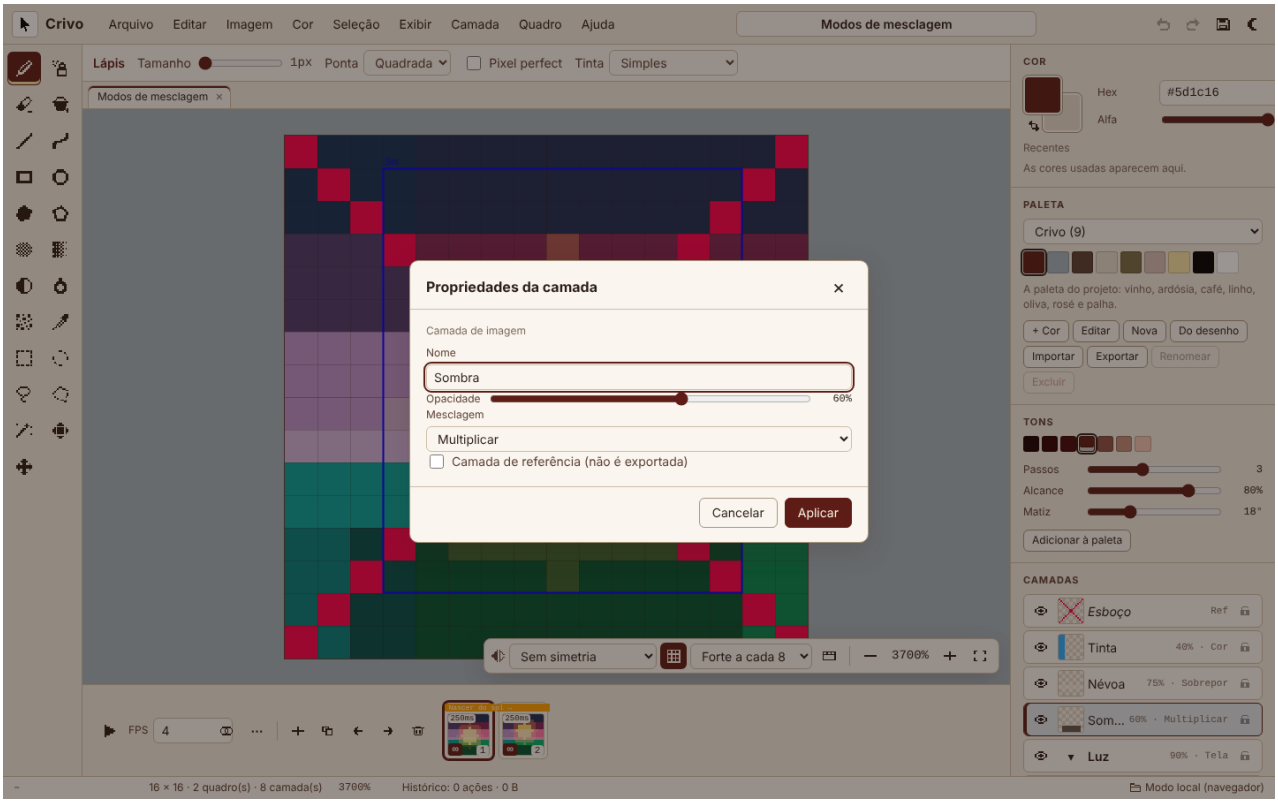
Um grupo junta camadas: o triângulo (▸/▾) abre e fecha a lista, e as camadas de dentro aparecem recuadas. Ocultar o grupo oculta tudo o que está nele, e a opacidade e o modo do grupo valem para o conjunto. Como no Aseprite, um grupo **normal a 100%** é "pass-through": as camadas de dentro se misturam direto com as de baixo. Com outro modo ou opacidade, o conteúdo do grupo é composto à parte e depois misturado como se fosse uma camada só.

Modos de mistura NOVO

São 19, escolhidos nas propriedades da camada. O modo normal segue exatamente a conta da 2.0; os outros seguem as fórmulas da especificação de composição do W3C (capítulo 13), iguais no editor, no servidor e na ferramenta em Java.

Modo	Nome no arquivo	Efeito
Normal	`normal`	A camada de cima cobre a de baixo, conforme o alfa (idêntico à 2.0)
Multiplicar	`multiply`	Escurece: multiplica as cores; branco não muda nada
Tela	`screen`	Clareia: o inverso do multiplicar; preto não muda nada
Sobrepor	`overlay`	Multiplica os tons escuros e clareia os claros da camada de baixo

Modo	Nome no arquivo	Efeito
Escurecer / Clarear	`darken / lighten`	Fica com o canal mais escuro / mais claro
Subexposição / Superexposição de cor	`color-dodge / color-burn`	Clareia / escurece a camada de baixo pela cor de cima
Luz forte / Luz suave	`hard-light / soft-light`	Sobrepor guiado pela camada de cima, forte ou suave
Diferença / Exclusão	`difference / exclusion`	Diferença entre as cores; a exclusão é mais suave
Matiz / Saturação / Cor / Luminosidade	`hue / saturation / color / luminosity`	Pegam só o matiz, a saturação, matiz e saturação, ou a luminosidade da camada de cima
Adição / Subtração / Divisão	`addition / subtract / divide`	Somam, subtraem ou dividem os canais (com limite em 0 e 1)



Propriedades da camada Sombra: nome, opacidade, modo de mistura (multiplicar) e referência.

Camadas de referência e de tiles NOVO

Uma **camada de referência** (um esboço, uma foto para copiar) aparece no editor, mas fica fora de todas as exportações: PNG, GIF, APNG, sprite sheet, dados, PNG do servidor e ferramenta em Java.

Camada → Nova camada de tiles... cria um tileset, com o tamanho do tile, e uma camada que guarda índices de tiles. O painel mostra os tiles, com o botão **+ Tile**. Em **Modo tile**, o lápis põe o tile escolhido, a borracha põe o tile 0 (vazio) e o conta-gotas pega o tile; em **modo pixel**, editar um tile atualiza todas as cópias dele. A grade dos tiles aparece no visor.

Quadros

Cada quadro tem o seu conteúdo em cada camada; os nomes, a ordem, a visibilidade e a opacidade das camadas valem para todos. O menu **Quadro** e os botões da linha do tempo criam um quadro vazio depois do atual (`Alt` + `N`), duplicam, movem e excluem. **Propriedades do quadro…** (ou duplo clique no quadro) define a duração própria, em milissegundos.

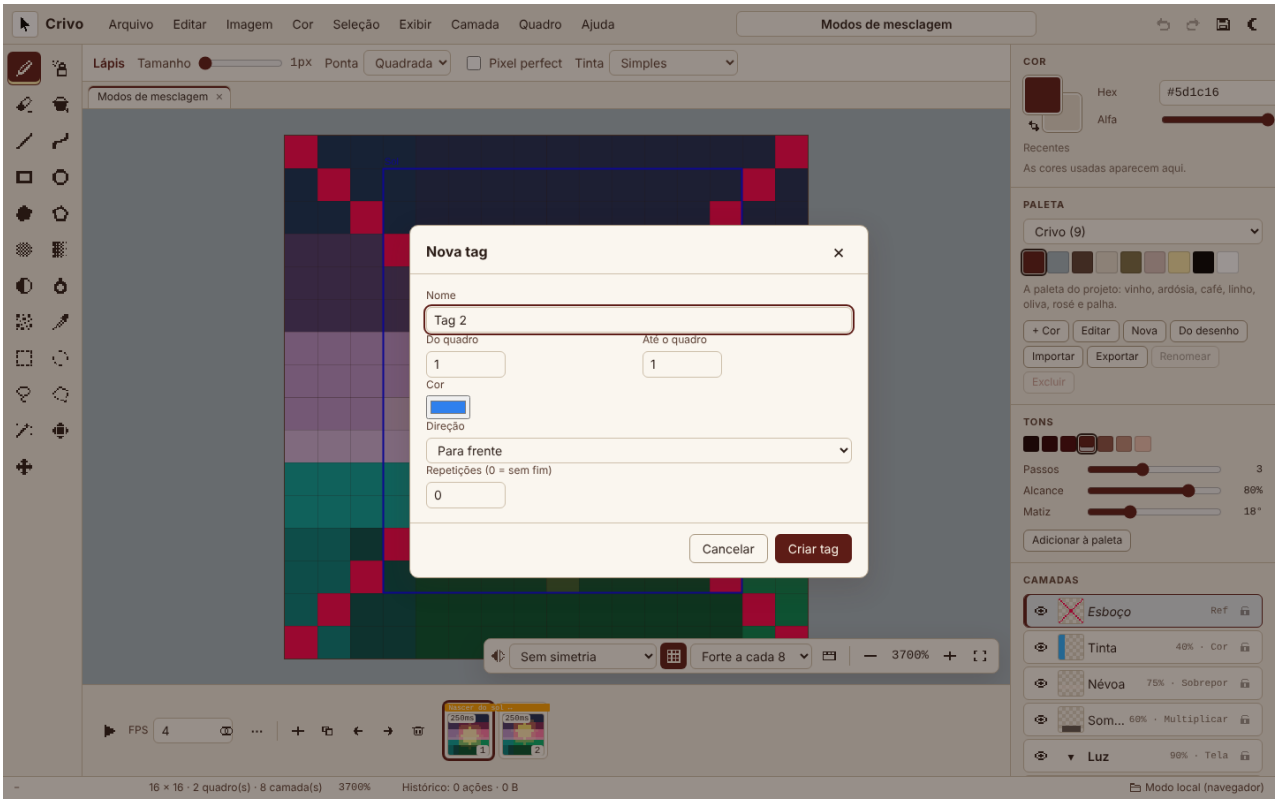
O campo FPS define a velocidade dos quadros sem duração própria (1 a 60 quadros por segundo). O botão tocar, ou `Enter`, repete a animação; parar volta ao quadro em que você estava. As teclas `,` e `.` andam entre os quadros.

Cels vinculados NOVO

Novo quadro vinculado cria um quadro que divide as células do atual, e **Vincular célula a quadros…** liga a célula da camada atual a outros quadros. Editar uma célula vinculada muda todas, e o "∞" na linha do tempo marca o vínculo. **Desvincular célula** faz uma cópia independente.

Tags NOVO

Quadro → Nova tag… marca um trecho da animação, com nome, cor, primeiro e último quadro, direção (para frente, para trás ou pingue-pongue) e repetições (0 = sem fim). As tags aparecem como faixas coloridas acima dos quadros; clique numa para editar. Durante a reprodução, o Crivo fica dentro da tag do quadro atual, na direção dela.



A tag Nascer do sol sobre os dois quadros e o diálogo Nova tag.

Papel cebola

Com o papel cebola ligado (`K`), os quadros vizinhos aparecem por baixo do atual, para você acompanhar o movimento. **Exibir → Configurar papel cebola…** NOVO, ou o botão ⋯ da linha do tempo, escolhe de 0 a 3 quadros

antes e depois e o estilo: normal ou tingido (vermelho antes, azul depois). Ele não aparece durante a reprodução nem é exportado.

Slices NOVO

Exibir → Editar slices… cria áreas nomeadas sobre a matriz, com cor, retângulo, centro de 9-slice e pivô opcionais, para botões, caixas de texto e pontos de origem de sprites. Elas vão para o `.aseprite` e para o JSON do Aseprite; **Exibir → Slices** mostra ou esconde o contorno delas.

Capítulo 9

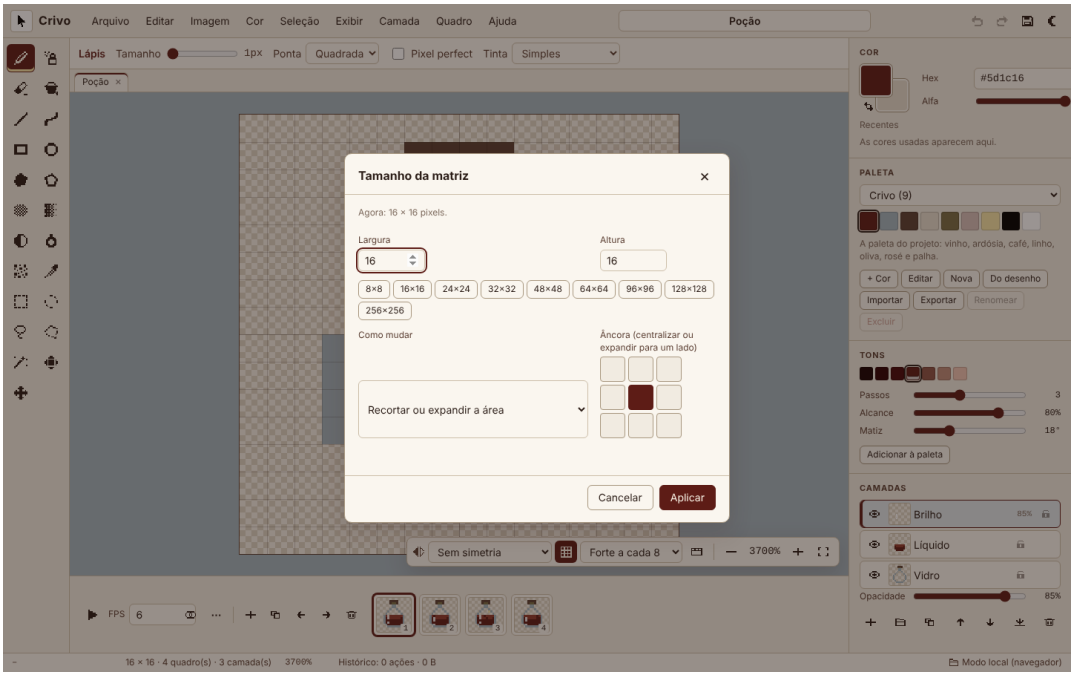
Tamanho da matriz, transformações e filtros

Em **Imagem → Tamanho da matriz** (`Ctrl` + `Alt` + `C`) você escolhe a nova largura e altura (1 a 1024), com atalhos para os tamanhos mais comuns, e um de dois modos:

Modo	O que acontece
Recortar ou expandir a área	Os pixels não mudam de tamanho. A âncora diz onde o desenho atual fica: no centro, a área cresce ou encolhe igualmente dos dois lados; num canto ou lado, cresce só na direção oposta
Escalar o desenho	O desenho é redimensionado por vizinho mais próximo: cada pixel novo copia o pixel original mais próximo do seu centro, sem criar cores novas

Os dois modos valem para todas as camadas e quadros e podem ser desfeitos. Girar 90° troca a largura pela altura.

Editar → Espelhar na horizontal (`Shift` + `H`) e **na vertical** (`Shift` + `V`) valem para a camada selecionada no quadro atual; com uma seleção ativa, espelham só a área selecionada, no próprio lugar, sem mexer no resto da camada.



Diálogo de tamanho da matriz com a âncora no centro.

Recortar e aparar

Comando	O que faz
Recortar para a seleção	A matriz passa a ter exatamente o tamanho da seleção, em todas as camadas e quadros. O que estava fora é descartado
Aparar bordas transparentes	Procura o menor retângulo que contém algum pixel visível em todas as camadas e quadros e corta o resto. Útil antes de exportar um sprite desenhado numa matriz grande

Filtros na camada

Os filtros atuam na camada selecionada, no quadro atual (a substituição de cor, em todos os quadros), e respeitam a seleção quando houver. Todos entram no histórico.

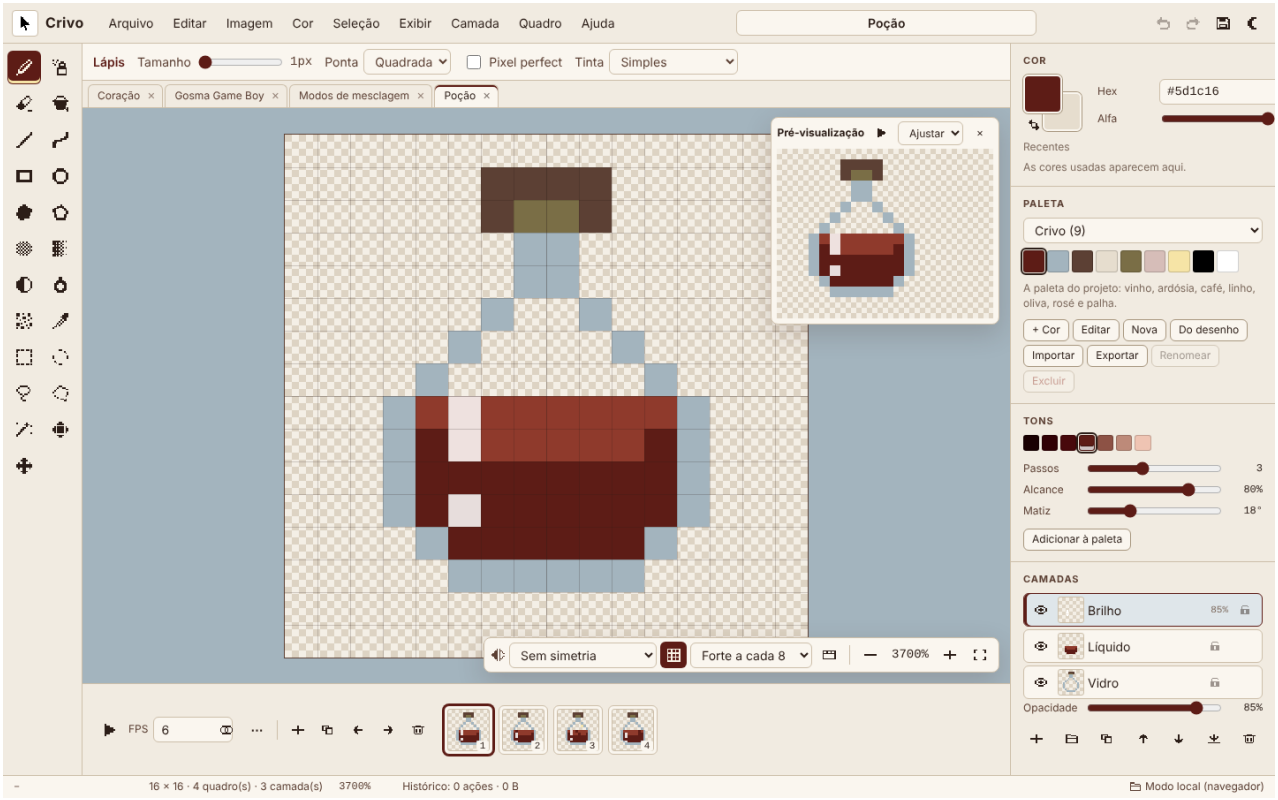
Comando	O que faz
Deslocar meia matriz (ladrilho)	Move a camada meia largura e meia altura com repetição: o que sai por um lado entra pelo outro. Atalho `Shift` + `T`
Adicionar contorno (4 ou 8 vizinhos)	Pinta com a cor principal todo pixel transparente que encosta num pixel opaco. Com 4 vizinhos o contorno é reto, só pelos lados; com 8 ele também fecha as diagonais
Substituir a cor primária pela secundária (todos os quadros)	Troca todo pixel exatamente igual à cor primária pela secundária, na camada atual e em todos os quadros da animação, respeitando a seleção. A troca inteira é um único passo no histórico. Ideal para mudar a cor de uma roupa ou de um olho numa animação sem repassar quadro a quadro
Converter para a paleta atual	Troca cada pixel opaco pela cor mais próxima da paleta selecionada, medindo a distância no espaço OKLab. Ótimo depois de importar uma imagem ou para trocar um sprite para a paleta do Game Boy
Inverter cores	Troca cada canal R, G e B por 255 menos ele, mantendo o alfa
Tons de cinza	Troca cada pixel por um cinza com a mesma luminosidade OKLab, mantendo o alfa

Capítulo 10

Importação e exportação

Abrir arquivos e abas NOVO

Arquivo → Abrir arquivo... (`Ctrl` + `O`) abre `.crivo`, `.json`, `.aseprite` e `.ase`, sempre numa **aba** nova acima da matriz; novo projeto, projetos salvos e importar imagem também abrem em abas. Arrastar um desses arquivos para a tela abre do mesmo jeito (imagens vão para o diálogo de importação). Clique numa aba para trocar de projeto, use o clique do meio ou **Fechar projeto (aba)** para fechar, e `Ctrl` + `Tab` e `Ctrl` + `Shift` + `Tab` para andar entre elas. A **Pré-visualização** (`F7`) mostra o quadro inteiro numa janela à parte, com o retângulo da parte visível; clicar ou arrastar nela move a visão, e ela tem a própria reprodução.



Quatro projetos abertos em abas e a janela de pré-visualização.

Arquivos do Aseprite NOVO

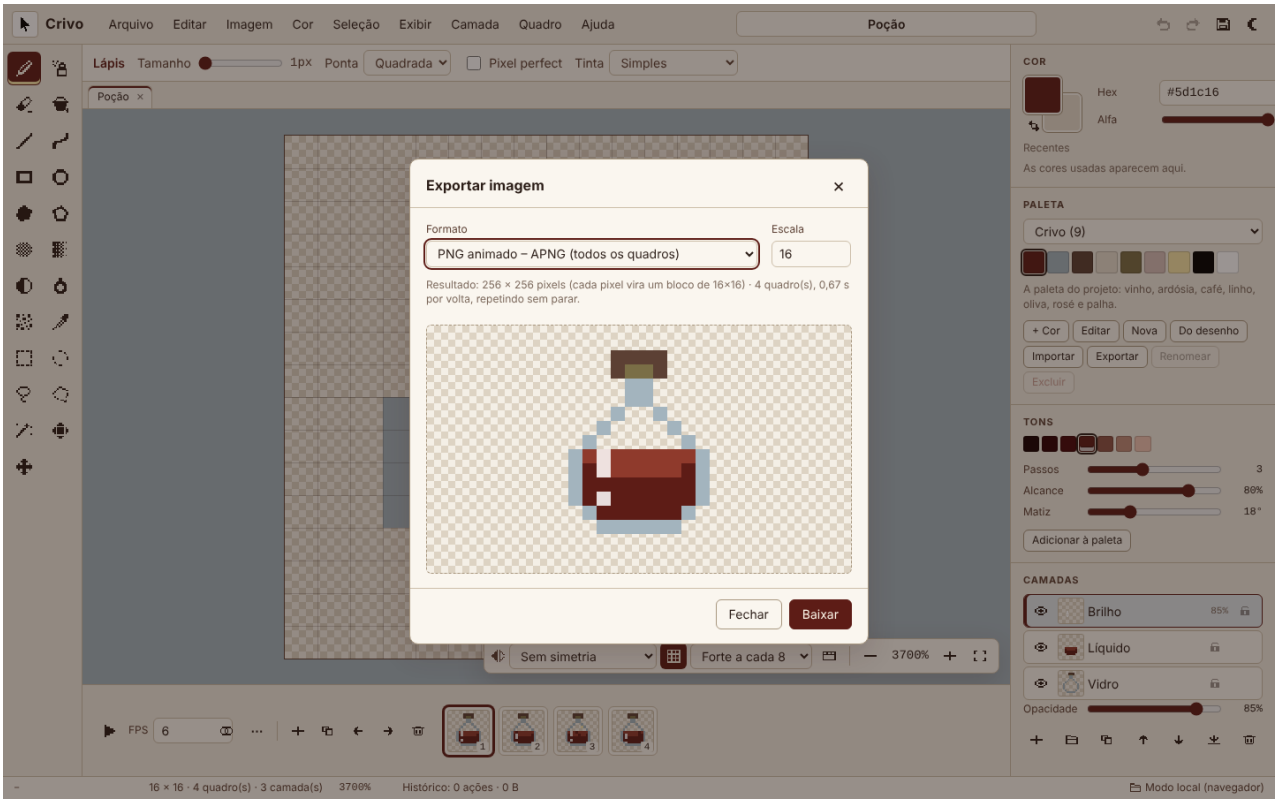
Sentido	O que vai junto
Abrir	Imagens em 32 bits (RGBA), 16 bits (tons de cinza) e 8 bits (indexado); camadas com visibilidade, bloqueio, opacidade, modo de mistura, grupos, grupos fechados e referência; cels normais, vinculados, comprimidos e de tilemap; paletas, tags, slices (a primeira chave) e tilesets
Exportar	**Arquivo → Exportar como .aseprite**: RGBA de 32 bits, cels comprimidos e vinculados, grupos, modos de mistura, opacidade, camadas de referência, tags, slices, paleta, duração de cada quadro e tilesets

O leitor e o gravador ficam em `js/core/aseprite.js`, escritos a partir da especificação pública do formato, com o próprio zlib (`js/core/zlib.js`) para os cels comprimidos.

Exportar imagem

Arquivo → Exportar imagem (PNG, JPEG ou GIF) (`Ctrl` + `E`) gera PNG ou JPEG de um quadro, ou um GIF animado com todos os quadros. Com escala 1, o arquivo tem exatamente o tamanho da matriz (pixel art pura). Com escala N, cada pixel vira um bloco de N×N, para não ficar minúsculo em redes sociais ou documentos. A ampliação é sempre por vizinho mais próximo.

O PNG mantém a transparência. O JPEG não tem transparência, então os pixels transparentes recebem a cor de fundo escolhida; a qualidade vai de 10 a 100%. Com o servidor, o mesmo PNG pode ser pedido por programas em `GET /api/v2/projects/{id}/png` (veja o capítulo da API).



Diálogo Exportar imagem com o PNG animado (APNG) escolhido.

GIF animado

Arquivo → Exportar GIF animado (`Ctrl` + `Shift` + `G`), ou a opção **GIF animado (todos os quadros)** do diálogo Exportar imagem, junta a animação inteira num único arquivo `.gif`, que abre em qualquer navegador, rede social ou aplicativo de mensagens. O codificador é do próprio Crivo, em `js/core/gif.js`, sem bibliotecas.

Aspecto	Como fica no GIF
Escala	De 1 a 64, por vizinho mais próximo; o GIF pode ter no máximo 4096 pixels de lado
Cores	Uma paleta global exata com até 255 cores, mais o índice 0 reservado para a transparência. Se a animação tiver mais de 255 cores, elas são aproximadas num cubo de 6 × 7 × 6 = 252 cores
Transparência	Pixel com alfa abaixo de 50% vira transparente; os demais ficam opacos (o GIF não tem transparência parcial)

Aspecto	Como fica no GIF
Tempo de cada quadro	A duração própria do quadro ou 1000 / FPS, convertida em centésimos de segundo, com mínimo de 2 (navegadores tratam valores menores como lentos)
Repetição	Infinita, pelo bloco `NETSCAPE2.0`, quando há mais de um quadro
Troca de quadro	Descarte "restaurar ao fundo": cada quadro começa limpo, então partes transparentes não deixam rastro do quadro anterior

PNG animado e quadros em .zip NOVO

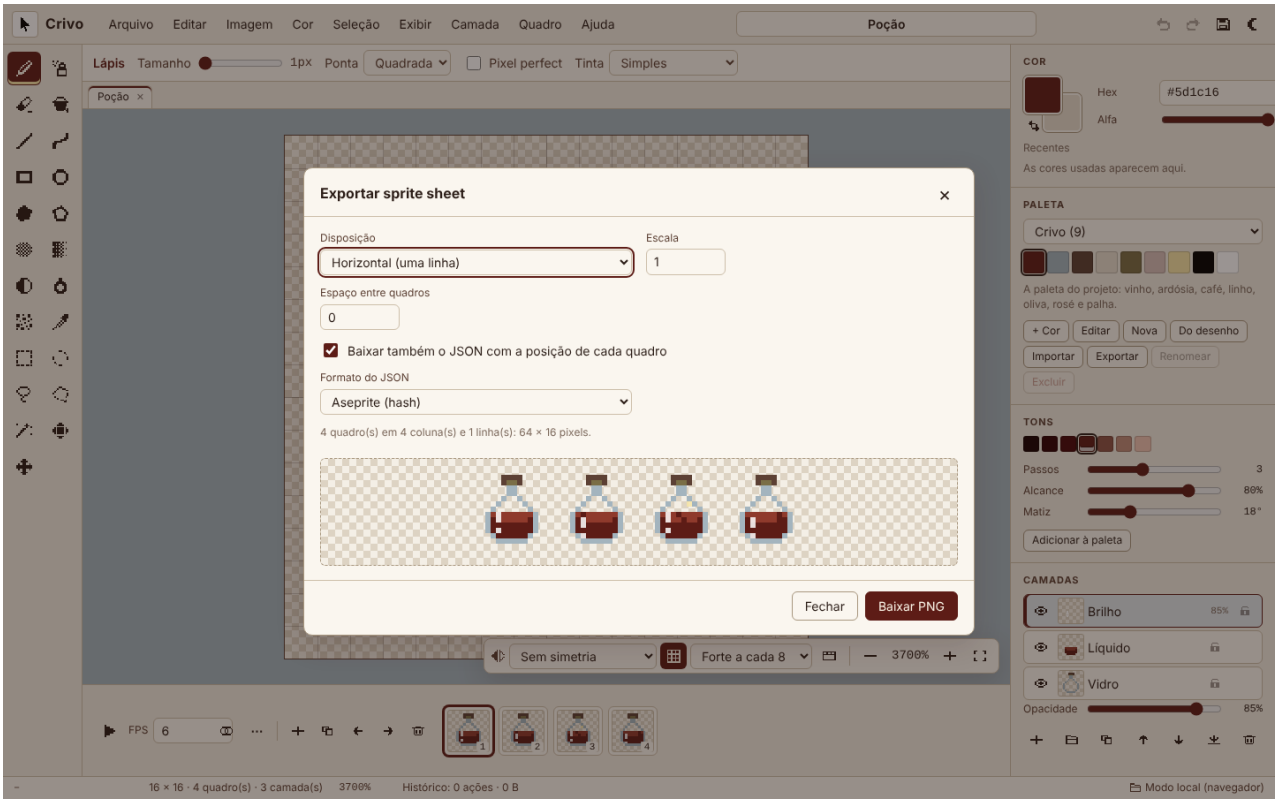
Arquivo → Exportar PNG animado (APNG)... gera um PNG animado, que abre em qualquer navegador atual, com a duração de cada quadro e transparência total (ao contrário do GIF, sem limite de 256 cores nem alfa de um bit). **Exportar quadros em PNG (.zip)...** gera um PNG por quadro, dentro de um `.zip`. Os dois usam a escala do diálogo e têm codificadores próprios (`js/core/apng.js` e `js/core/zip.js`). Camadas de referência ficam fora de todas as exportações.

Sprite sheet

Arquivo → Exportar sprite sheet junta todos os quadros numa única imagem PNG, em linha (horizontal), coluna (vertical) ou grade, com escala e espaço entre os quadros. Junto vai um JSON com a posição e a duração de cada quadro, no formato que a maioria dos motores de jogo sabe ler:

```
{
  "name": "heart-sprites",
  "image": "heart-sprites.png",
  "width": 64, "height": 16, "fps": 8,
  "frames": [
    { "index": 0, "x": 0, "y": 0, "width": 16, "height": 16, "duration": 240 },
    { "index": 1, "x": 16, "y": 0, "width": 16, "height": 16, "duration": 125 }
  ]
}
```

O campo **Formato do JSON** NOVO troca esse JSON pelo do Aseprite, em **hash** (quadros por nome) ou **array**, com `frames` e `meta` (`frameTags`, `layers` e `slices`): o arquivo entra direto nos motores e ferramentas que já leem o JSON exportado pelo Aseprite.



Sprite sheet em linha, com o JSON no formato hash do Aseprite.

Exportar dados da matriz

Arquivo → Exportar dados (`Ctrl` + `Shift` + `E`) transforma a imagem em código. Escolha o formato, os valores e os quadros; a prévia atualiza na hora e os botões copiam ou baixam o arquivo.

Valores	Como fica cada pixel
Índices de paleta	Um número que aponta para a lista `palette`; −1 é transparente (o valor de `transparent`). Opcionalmente, segue a ordem da paleta atual
Cores em hexadecimal	O código da cor, como `"#5d1c16"`; transparente vira `null` (`None` em Python)

Formato	Arquivo	Estrutura
JSON	`.json`	Objeto com `width`, `height`, `palette` e `frames[frame][row][column]`
CSV	`.csv`	Uma linha da matriz por linha do arquivo; quadros separados por uma linha em branco
JavaScript	`.js`	`export const name = {...}` pronto para `import`
Python	`.py`	`NAME = {...}` com listas aninhadas
C	`.h`	Cabeçalho com `#define` das dimensões, paleta `uint32_t` (RGBA) e matriz `int16_t` (ou `int32_t` com mais de 32.767 cores) ou `uint32_t`
Java	`.java`	Classe final com `WIDTH`, `HEIGHT`, `PALETTE` (ARGB) e `int[][][] FRAMES`; o arquivo tem o nome da classe em PascalCase (`Pocao.java`), como o `javac` exige

Classes Java grandes

O Java limita o bytecode de cada método a 64 KB, e uma matriz grande escrita como literal (`{ {0, 1, ...} }`) estoura esse limite: a classe não compila. Acima de 4.096 valores (quadros × altura × largura) ou de 2.048 cores, o Crivo troca o literal por texto compacto, que não tem esse limite:

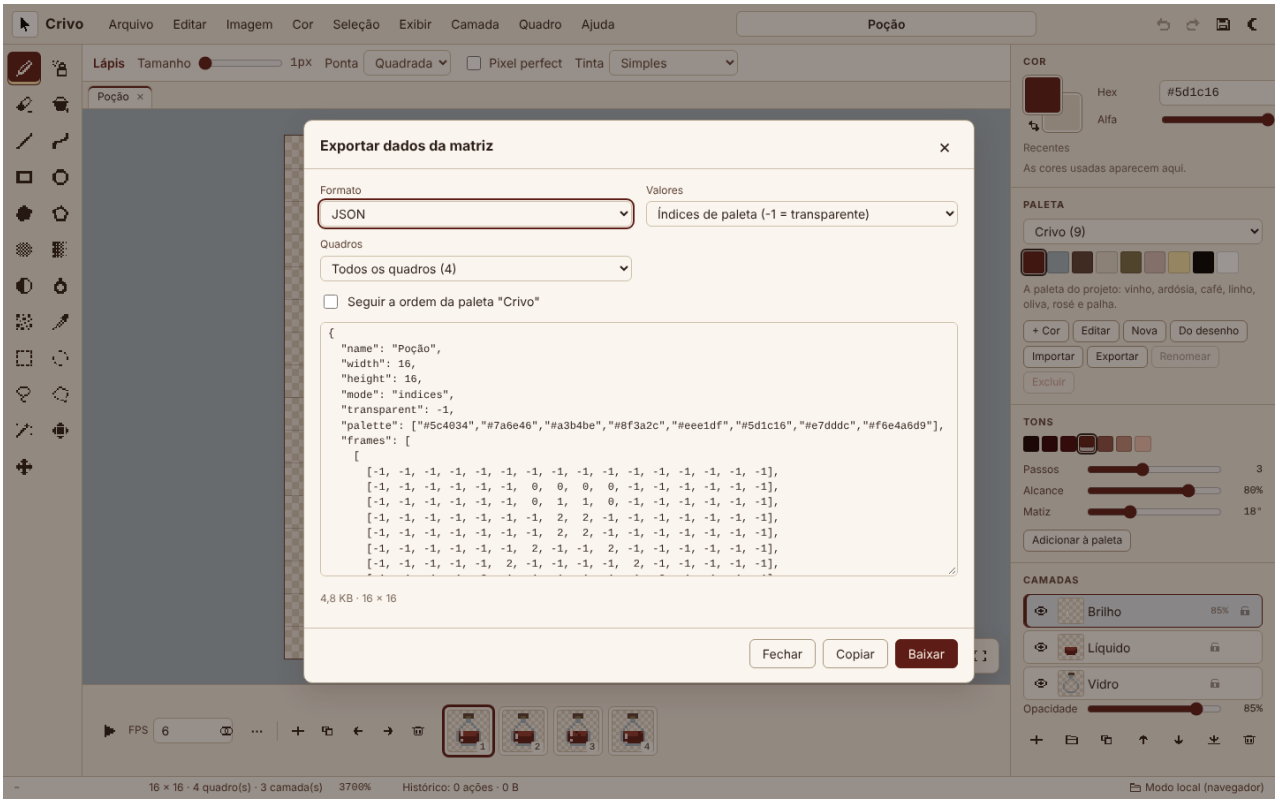
- os pixels vão em `private static final String[] DATA`, em pedaços de 20.000 caracteres: um caractere por pixel (índice + 33, e espaço para −1, o transparente) ou, com mais de 65.502 cores ou nos modos hexadecimal e ARGB, dois caracteres por valor (os 16 bits altos e os 16 baixos);
- a paleta vai em `PALETTE_DATA`, com dois caracteres por cor;
- um bloco `static` lê tudo quando a classe é carregada e preenche `PALETTE` e `FRAMES`, então quem usa a classe não percebe diferença.

O editor, a API e o comando `crivo.sh matrix` geram o mesmo formato. Sprites pequenos continuam com a matriz legível, como antes.

Desde a 1.1.0, os seis formatos também saem pela API, em `/export?format=json|csv|js|py|c|java`. Um uso típico em JavaScript, desenhando o sprite exportado numa tela do jogo:

```
import { pocao } from './pocao.js';

const ctx = document.querySelector('canvas').getContext('2d');
const frame = pocao.frames[0];
frame.forEach((row, y) => row.forEach((index, x) => {
  if (index < 0) return;
  ctx.fillStyle = pocao.palette[index];
  ctx.fillRect(x * 4, y * 4, 4, 4);
}));
```



Exportação dos quatro quadros da poção em JSON, com índices de paleta.

Importar imagens

Arquivo → Importar imagem ou sprite sheet aceita PNG, JPEG, GIF (primeiro quadro), WebP e BMP de até 4096 pixels de lado. Ao abrir o arquivo, o Crivo procura o **tamanho do pixel**: muitas imagens de pixel art circulam ampliadas, e cada pixel original virou um bloco N×N. Ele mede o comprimento de todas as sequências de cores iguais nas linhas e colunas e calcula o máximo divisor comum delas; se o resultado for 4, cada pixel original ocupa 4×4 e a imagem é reduzida 4 vezes, pegando o pixel do centro de cada bloco.

Abrir como	Resultado
Novo projeto	Um projeto com o tamanho da imagem reduzida
Nova camada	Uma camada nova no quadro atual; o que passar da matriz é cortado
Animação	A imagem é fatiada em quadros do tamanho informado, da esquerda para a direita e de cima para baixo, ignorando os pedaços vazios

Uma foto ou uma imagem com muitas cores pode ir direto para uma paleta fechada com **Imagem → Converter para a paleta atual**.

O arquivo .crivo

Arquivo → Baixar arquivo .crivo salva o projeto inteiro num JSON legível: nome, tamanho, FPS, paleta, camadas, quadros e o conteúdo de cada célula em RLE codificado em base64. **Abrir arquivo .crivo** (`Ctrl` + `O`) faz o caminho de volta. É o formato que a API recebe e devolve e que a ferramenta em Java lê. Na 2.0.0, o formato passou para a versão 2, com as chaves em inglês; o RLE das células não mudou:

```
{
  "format": "crivo", "version": 2, "name": "Poção",
  "width": 16, "height": 16, "fps": 6,
  "palette": ["#5d1c16", "#a3b4be", "#5c4034"],
  "layers": [{ "id": "c...", "name": "Vidro", "visible": true, "opacity": 100, "locked": false }],
  "frames": [{ "id": "q...", "duration": null }],
  "cells": [{ "frame": "q...", "layer": "c...", "rle": "EAAAAAAEXEBANP8..." }]
}
```

Arquivos da versão 1, salvos pela 1.x com as chaves em português, continuam abrindo no editor, na API e na ferramenta em Java: são convertidos na leitura, e o próximo salvamento já grava a versão 2. Um arquivo com `version` maior que 2 foi salvo por um Crivo mais novo e é recusado com um aviso.

Chaves opcionais da 2.1.0 NOVO

A 2.1.0 continua gravando a versão 2. Os recursos novos usam chaves opcionais, gravadas só quando diferem do padrão e ignoradas por leitores antigos; um arquivo da 2.0 abre e salva igual.

Onde	Chave	Valor
Raiz	`colorMode`	`rgba` (padrão), `indexed` ou `grayscale`
Raiz	`tags`	Lista de `{id, name, from, to, direction, repeat, color}`; `from` e `to` são ids de quadros e `direction` é `forward`, `reverse` ou `pingpong`

Onde	Chave	Valor
Raiz	`slices`	Lista de `{id, name, color, x, y, width, height, center, pivot}`; `center` e `pivot` podem ser `null`
Raiz	`tilesets`	Lista de `{id, name, tileWidth, tileHeight, tiles}`; cada tile em RLE base64, e o tile 0 é o vazio
Camada	`blendMode`, `kind`, `parent`, `collapsed`, `tileset`, `reference`	Modo de mistura; `image`, `group` ou `tilemap`; id do grupo; grupo fechado; id do tileset; camada de referência
Célula	`link`, `tiles`	Id do quadro da célula de origem e o RLE dos índices de tiles; o `rle` continua com os pixels, para que leitores da 2.0 vejam a mesma imagem

Capítulo 11

Salvando o trabalho

Servidor ou navegador

Ao abrir, o editor chama `/api/v2/health`. Se a API responder, os projetos e as paletas vão para o banco do servidor; se não, ficam no navegador. A barra de status sempre mostra qual dos dois está em uso.

	Servidor Django	Navegador
Onde ficam os projetos	PostgreSQL (ou SQLite)	IndexedDB, banco `crivo`, loja `projects`
Paletas suas	Tabelas `palette` e `palette_color`	`localStorage`, chave `crivo.palettes`
Versões anteriores	Sim, as últimas 20 por projeto	Não
Abrir em outro computador	Sim, pelo endereço do servidor	Não; use o arquivo `.crivo`
Rascunho automático	Sim	Sim

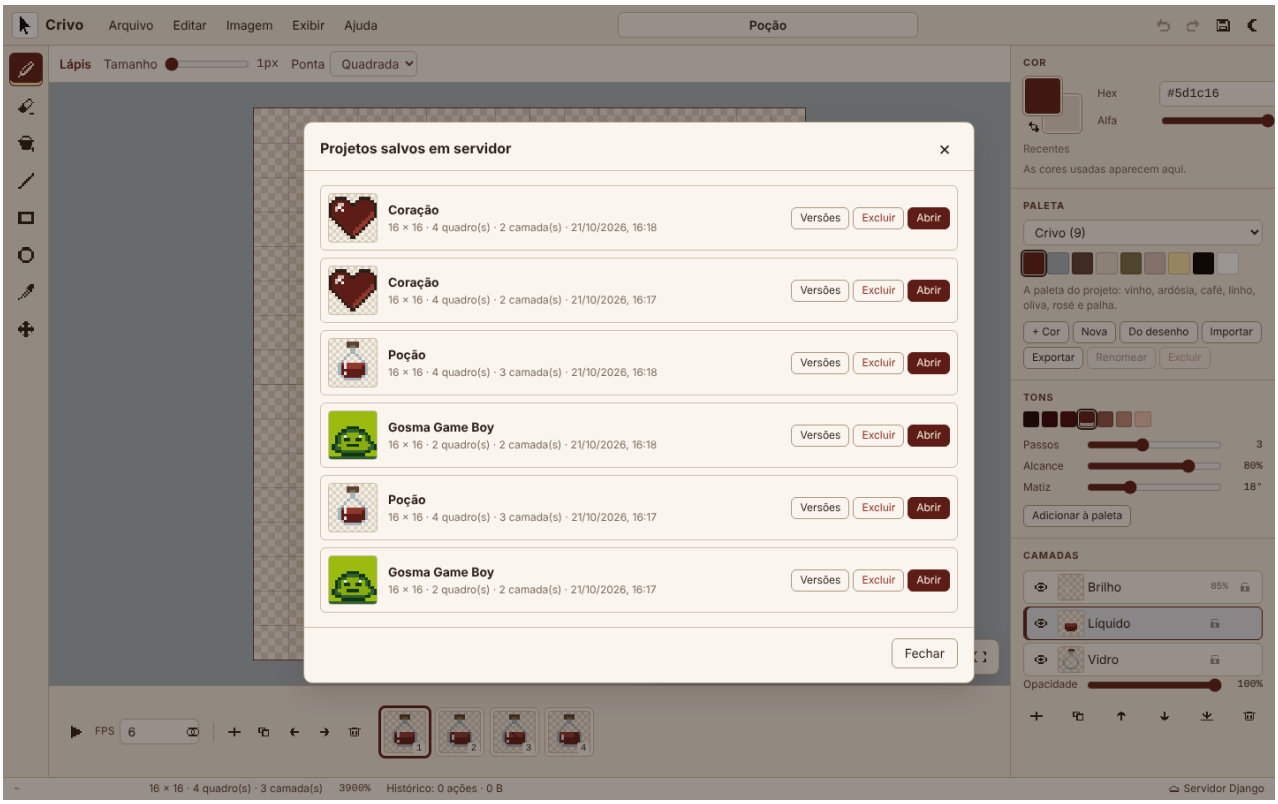
Na 2.0.0, as lojas do IndexedDB e as chaves do `localStorage` ganharam nomes em inglês. Na primeira abertura, o editor copia e converte os projetos, o rascunho, as paletas e as preferências guardados pela 1.x, sem perder nada.

Salvar, versões e conflitos

`Ctrl` + `S` salva o projeto atual. A primeira vez cria o projeto; as próximas substituem o conteúdo, e a versão anterior vai para o histórico de versões. **Salvar como cópia** cria um projeto novo. Em **Projetos salvos**, o botão **Versões** lista as versões guardadas e **Restaurar** volta para uma delas (a versão atual também fica guardada, então nada se perde).

Cada projeto tem um número de revisão. Se outra aba ou pessoa salvou o mesmo projeto depois que você o abriu, o servidor responde com conflito e o Crivo pergunta se você quer salvar por cima. A versão da outra pessoa continua no histórico de versões.

Se muitos salvamentos chegarem do mesmo endereço em pouco tempo, o servidor responde 429 e o Crivo avisa para tentar de novo em alguns segundos. O limite padrão (120 alterações por minuto) fica muito acima do uso normal; ele existe para conter scripts com defeito ou abuso.



Projetos salvos no servidor, com miniatura, tamanho, quadros e camadas.

Rascunho automático

Pouco mais de um segundo depois de cada mudança, e também ao fechar a aba, o projeto aberto é copiado para o IndexedDB. Na próxima vez que o Crivo abrir naquele navegador, o rascunho volta exatamente como estava, inclusive com a marca de mudanças não salvas (o ponto antes do nome na aba).

Capítulo 12

Por dentro: matemática e estruturas de dados

A matriz de pixels

Cada célula (um quadro de uma camada) é um `Uint32Array` com largura × altura posições. O pixel (x, y) fica no índice `y × largura + x`. Cada inteiro de 32 bits guarda os quatro canais; em máquinas little-endian, que são praticamente todas, os bytes na memória ficam na ordem R, G, B, A, a mesma do `ImageData.data`. Por isso a composição final é escrita direto num `Uint32Array` que enxerga o buffer do `ImageData`, sem conversão.

```
cor = (A << 24) | (B << 16) | (G << 8) | R
```

Composição das camadas

As camadas visíveis são combinadas de baixo para cima com o operador "over" em alfa direto, multiplicando o alfa de cada pixel pela opacidade da camada:

$$\alpha s = \alpha_frente \times opacidade \qquad \alpha o = \alpha s + \alpha_fundo \times (1 - \alpha s)$$

$$C = (C_frente \times \alpha s + C_fundo \times \alpha_fundo \times (1 - \alpha s)) / \alpha o$$

Pixels totalmente opacos numa camada a 100% são copiados direto e pixels transparentes são pulados, então o caso comum da pixel art custa uma comparação por pixel.

Linhas: algoritmo de Bresenham

Linhas, o traço do lápis entre dois eventos do mouse e as formas com pincel usam a versão inteira de Bresenham: um erro acumulado decide, a cada passo, se a linha anda em x, em y ou nos dois, sem multiplicação nem ponto flutuante. O resultado tem exatamente um pixel por coluna (ou linha) no eixo dominante, sem buracos e sem degraus duplos.

```
dx = |x1 − x0|;  dy = −|y1 − y0|;  erro = dx + dy
repita:
    pinta(x0, y0); se chegou: pare
    e2 = 2 × erro
    se e2 ≥ dy: erro += dy; x0 += sx
    se e2 ≤ dx: erro += dx; y0 += sy
```

Pixel perfect: cantos em L

O filtro guarda os últimos pixels do traço. Quando chega um ponto novo C, ele olha os dois anteriores A e B: se B encosta em A e em C só pelos lados (um na horizontal, o outro na vertical) e A e C são vizinhos na diagonal, B é um canto em L e volta à cor que tinha antes do traço.

$$canto(A, B, C) = (|A{-}B| + |B{-}C| = 2 \text{ passos ortogonais}) \text{ e } |Ax - Cx| = |Ay - Cy| = 1$$

Como o teste só olha três pontos, ele roda em tempo constante por evento do mouse, e o histórico continua guardando só os pixels que mudaram de verdade.

Elipses: ponto médio na caixa

A elipse usa a variante de Alois Zingl do algoritmo do ponto médio, que trabalha com a caixa (x0, y0)–(x1, y1) em vez de centro e raios. Isso permite elipses e círculos de diâmetro par, que não têm um pixel central. O algoritmo percorre um quadrante atualizando incrementos de erro inteiros e espelha cada ponto nos outros três. No modo preenchido, cada linha da elipse vira uma faixa horizontal.

Balde de tinta: preenchimento por varredura

A versão ingênua do flood fill empilha os quatro vizinhos de cada pixel e chega a milhões de entradas na pilha. O Crivo usa preenchimento por varredura (*scanline*): ao tirar uma semente da pilha, ele expande para a esquerda e a direita até a borda, marca a faixa inteira e, nas linhas de cima e de baixo, empilha só uma semente por trecho contínuo que ainda precisa ser pintado. Com 8 vizinhos, o trecho examinado avança um pixel para cada lado, alcançando as diagonais.

Um vetor `Uint8Array` de visitados evita reprocessar pixels. A tolerância compara cada pixel com a cor do ponto clicado pela maior diferença entre os canais; todos os pixels totalmente transparentes contam como a mesma cor. Com uma seleção ativa, os pixels de fora são descartados antes de pintar.

Pontilhado ordenado: a matriz de Bayer

A matriz de Bayer de ordem n distribui os números de 0 a $n^2 - 1$ de forma que valores próximos fiquem o mais longe possível uns dos outros. Ela é construída por recorrência a partir da de 2×2:

$$B_2 = \begin{vmatrix} 0 & 2 \\ 3 & 1 \end{vmatrix} \qquad B_{2n} = \begin{vmatrix} 4 \cdot B_n + 0 & 4 \cdot B_n + 2 \\ 4 \cdot B_n + 3 & 4 \cdot B_n + 1 \end{vmatrix}$$

Cada pixel (x, y) ganha um limiar entre 0 e 1 a partir da célula da matriz que cai sobre ele, repetida pela matriz inteira:

$$\text{limiar}(x, y) = (B[y \bmod n][x \bmod n] + 0{,}5) / n^2$$



No **pincel de pontilhado**, o pixel recebe a cor principal quando `limiar < densidade` e a secundária no caso contrário. Como o limiar depende só de (x, y), o padrão é o mesmo em toda a matriz e traços sobrepostos dão exatamente o mesmo resultado.

Degradê: projeção e distância

O degradê calcula, para cada pixel, um valor t entre 0 (cor principal) e 1 (cor secundária) e o compara com o limiar de Bayer daquele pixel: fica a secundária se `t > limiar`. No modo linear, t é a projeção do pixel sobre o vetor do arraste; no radial, é a distância ao ponto inicial dividida pelo comprimento do arraste. Os dois são limitados ao intervalo [0, 1].

$$\text{linear: } t = ((P - A) \cdot (B - A)) / |B - A|^2 \qquad \text{radial: } t = |P - A| / |B - A|$$

Com Bayer 4×4 há 17 proporções possíveis de mistura (de 0 a 16 pixels secundários em cada bloco 4×4); com 8×8, 65. O ponto P é o centro do pixel, (x + 0,5, y + 0,5), para que o degradê fique simétrico.

GIF: compressão LZW

O GIF guarda cada quadro como uma sequência de índices da paleta, comprimida com LZW (Lempel–Ziv–Welch). A ideia é montar, durante a compressão, um dicionário de sequências que já apareceram, e escrever o código da maior sequência conhecida em vez dos índices um a um. Quem lê o arquivo reconstrói o mesmo dicionário na mesma ordem, então ele nunca precisa ser gravado.

1. O dicionário começa com um código para cada cor (2^n códigos, com n de 2 a 8 conforme o tamanho da paleta), mais o código de limpeza (2^n) e o de fim ($2^n + 1$).
2. A sequência atual começa com o primeiro índice. Para cada índice seguinte, o codificador procura no dicionário o par (prefixo, próximo índice), em que o prefixo é o código da sequência atual.
3. Se o par existe, a sequência cresce e vira esse código. Se não existe, o codificador escreve o código da sequência atual, cria um código novo para o par e recomeça a sequência a partir do próximo índice.
4. Os códigos são escritos com n + 1 bits no começo. Quando o próximo código novo chega a 2^{bits}, o tamanho cresce um bit, até o máximo de 12 bits (4096 códigos).
5. Ao chegar a 4096 entradas, o codificador escreve o código de limpeza, o dicionário volta ao estado inicial e o tamanho volta a n + 1 bits.

```
dicionário[(prefixo, índice)] = próximo_código        bits = [log₂(próximo_código + 1)], de n + 1 até 12
```

Com isso, os códigos variam de 3 a 12 bits. Pixel art comprime muito bem: uma linha de fundo transparente vira poucos códigos que representam sequências cada vez maiores. Os bits são empacotados do menos para o mais significativo e cortados em sub-blocos de até 255 bytes, como manda o formato GIF89a. Um teste de ida e volta comprime e descomprime uma imagem grande o bastante para passar de 4096 códigos, para garantir que a limpeza do dicionário funciona.

RLE com varint

Pixel art tem longas sequências de cores iguais, principalmente transparência. Cada célula é guardada como uma lista de pares (quantidade, cor): a quantidade em varint (7 bits por byte, o bit mais alto indica que há mais bytes) e a cor em 4 bytes RGBA. A codificação faz duas passadas: a primeira conta o tamanho exato e a segunda escreve num `Uint8Array` já do tamanho certo.

Célula	Bruta	Em RLE
32×32 vazia	4.096 bytes	6 bytes
16×16 da poção (vidro)	1.024 bytes	225 bytes
256×256 com fundo de uma cor e um desenho no meio	262.144 bytes	alguns KB

Histórico ilimitado

O histórico é uma pilha de passado e uma de futuro. O que cada entrada guarda depende do tamanho da mudança, para gastar o mínimo de memória:

- **Traço:** durante o arraste, um `Map` guarda a cor original de cada pixel tocado pela primeira vez. Ao soltar o mouse, só os pixels que realmente mudaram viram três vetores: índices, cores antes e cores depois (12 bytes por pixel). Mover pixels, colar, o degradê e os filtros usam o mesmo mecanismo.
- **Célula:** se mais de 2.048 pixels mudaram (um balde numa área grande, por exemplo), guardar a célula inteira antes e depois em RLE costuma ser menor que a lista de índices.
- **Estado:** mudanças de estrutura (camadas, quadros, tamanho, recorte, aparo, giro, opacidade, nome) guardam um instantâneo do documento com todas as células em RLE, e células vazias nem entram.

Desfazer aplica o lado "antes" da entrada e a move para o futuro; refazer faz o contrário; uma ação nova descarta o futuro. Não há limite de passos, e o total ocupado aparece na barra de status.

Cores em OKLab e OKLCH

Para gerar tons, a cor sRGB passa para RGB linear (removendo a curva gama), depois para o espaço LMS dos cones por uma matriz 3×3, tira-se a raiz cúbica de cada canal e uma segunda matriz chega ao OKLab (L, a, b). Em coordenadas polares, OKLCH tem luminosidade L, croma $C = \sqrt{a^2 + b^2}$ e matiz h = atan2(b, a).

```
L_sombra = L − (L − 0,08) × t        L_luz = L + (0,985 − L) × t        t = (i / passos) × alcance
```

Cada passo i também gira o matiz em direção a 275° (sombras) ou 95° (luzes), pelo menor arco, e reduz o croma nas luzes para que elas não fiquem saturadas demais. Se a cor resultante sair da gama sRGB, uma busca binária de 24 passos reduz o croma até ela caber.

Conversão para a paleta e sombreamento

Para levar um pixel à cor mais próxima da paleta, o Crivo compara as cores no OKLab, em que a distância euclidiana acompanha bem a diferença que os olhos percebem (em RGB, dois verdes podem parecer mais distantes que um verde e um cinza). As cores da paleta são convertidas uma vez, e cada cor diferente do desenho é calculada uma vez só, guardada num `Map`:

```
d(c, p) = (L_c − L_p)² + (a_c − a_p)² + (b_c − b_p)²        escolhida = argmin d(c, p)
```

O clarear e escurecer no modo rampa usa o mesmo cálculo: a paleta é ordenada por L, o pixel vai para a cor mais próxima e anda uma posição na ordem. No modo luminosidade, o pixel vai para OKLCH, recebe L ± intensidade (limitado a [0, 1]) e volta para sRGB com a mesma redução de croma do gerador de tons. Tons de cinza usam o L do OKLab com a = b = 0.

Contorno por vizinhança

O contorno lê uma cópia da célula e, para cada pixel transparente, olha os vizinhos: os 4 dos lados (N, S, L, O) ou também as 4 diagonais. Se algum for opaco, o pixel recebe a cor principal. Ler da cópia, e não da célula que está sendo pintada, garante que o contorno tenha exatamente 1 pixel e não "cresça" sobre ele mesmo.

```
contorno(x, y) ⇔ α(x, y) = 0  e  ∃ (dx, dy) ∈ vizinhos : α(x + dx, y + dy) > 0
```

Escala por vizinho mais próximo

Para ampliar por um fator inteiro N, cada linha da origem é expandida uma vez (cada pixel vira N pixels iguais, com `fill`) e copiada N vezes com `set`. Para redimensionar para qualquer tamanho, cada pixel de destino (x, y) busca a origem em ⌊(x + 0,5) × L_origem / L_destino⌋, o que mantém a imagem centrada e nunca cria cores novas.

Deslocamento com repetição e aparo

O deslocamento de meia matriz copia cada pixel (x, y) para ((x + dx) mod L, (y + dy) mod A), com dx = ⌊L / 2⌋ e dy = ⌊A / 2⌋, então nada se perde e dois deslocamentos seguidos numa matriz par voltam ao original. O aparo percorre a composição de todos os quadros (incluindo camadas ocultas) e guarda os menores e maiores x e y com alfa maior que zero; a matriz é então recortada nesse retângulo.

Capítulo 13

Arquitetura do código

Módulos do editor

Arquivo	Responsabilidade
`js/core/colors.js`	Empacotar e desempacotar RGBA, hexadecimal, HSL, OKLab, OKLCH, gerador de tons, sobreposição alfa
`js/core/geometry.js`	Bresenham, retângulo, elipse, pincel, espelhos da simetria, restrição de ângulo, detecção do tamanho do pixel, filtro pixel perfect
`js/core/fill.js`	Região do balde por varredura e preenchimento
`js/core/selection.js`	Retângulo normalizado, recorte da seleção ao documento, extrair e colar regiões, limites do conteúdo (aparar) e deslocamento com repetição
`js/core/filters.js`	Contorno, conversão para a paleta, inverter, tons de cinza, passo de luminosidade e passo na rampa da paleta
`js/core/dither.js`	Matrizes de Bayer 4×4 e 8×8, limiar por pixel, cor do pontilhado e os valores t dos degradês linear e radial
`js/core/gif.js`	Codificador de GIF89a: paleta global com transparência (ou cubo de 252 cores), LZW de 3 a 12 bits, atrasos, repetição e descarte
`js/core/rle.js`	RLE com varint e base64 próprio
`js/core/document.js`	`PixelDocument`: camadas, quadros, células, composição, redimensionar, escalar, recortar, girar, instantâneos; na 2.1.0, grupos, modos de mistura, referência, cels vinculados, tags, slices e tilesets
`js/core/history.js`	`History` e `Stroke` (traço), entradas de pixels, de célula e de estado, desfazer e refazer
`js/core/format.js`	Serializar e desserializar o `.crivo` (`serialize` e `deserialize`), convertendo arquivos da versão 1
`js/core/export.js`	Matrizes, geradores de JSON, CSV, JS, Python, C e Java, layout de sprite sheet, escala
`js/core/palettes.js`	Paletas predefinidas, importação, exportação e extração
`js/core/blend.js` NOVO	Os 19 modos de mistura, os nomes em português (BLEND_LABELS) e a composição de grupos
`js/core/tilemap.js` NOVO	Tilesets, índices de tiles e o desenho dos tiles nos pixels da célula
`js/core/animation.js` NOVO	Tags, direção, repetições e a ordem de reprodução
`js/core/brush.js` NOVO	Pincel personalizado, spray, desfocar, embaralhar e modos de tinta
`js/core/transform.js` NOVO	Rotação com RotSprite, escala e espelho da seleção
`js/core/colormode.js` NOVO	Modos de cor RGBA, indexado (com pontilhado) e tons de cinza

Arquivo	Responsabilidade
`js/core/adjust.js` NOVO	Brilho e contraste, matiz/saturação/luminosidade, curvas, posterizar, remover ruído e convolução
`js/core/palette-tools.js` NOVO	Ordenar, inverter, tirar duplicadas, rampa e redimensionar paletas
`js/core/aseprite.js` NOVO	Leitor e gravador do formato .aseprite
`js/core/apng.js, zip.js, zlib.js` NOVO	PNG animado, arquivo .zip e compressão zlib próprios
`js/ui/editor.js`	Estado do editor, seleção, área de transferência, eventos, cores, transações de traço e de estrutura, reprodução
`js/ui/viewport.js`	Canvas: composição, zoom, pan, xadrez, grade configurável, modo ladrilho, eixos, seleção tracejada, prévia das formas e cursor
`js/ui/tools.js`	Ponteiro, teclado e o comportamento de cada uma das 23 ferramentas, com polígonos, curvas e modos de seleção
`js/ui/panels.js`	Cor, paleta, tons, camadas (com grupos, modos e tiles) e linha do tempo (com tags e vínculos)
`js/ui/dialogs.js`	Novo, tamanho, exportações, importação, projetos, versões, atalhos e sobre
`js/ui/storage.js`	Cliente da API e armazenamento no IndexedDB (com a migração dos dados da 1.x)
`js/ui/layer-dialogs.js` NOVO	Propriedades de camada e quadro, camada de tiles, vínculos, tags, slices e papel cebola
`js/ui/palette-editor.js, color-picker.js` NOVO	Editor de paleta e seletor HSV com alfa
`js/ui/adjustments.js` NOVO	Diálogo dos ajustes com prévia ao vivo
`js/ui/tabs.js, minimap.js` NOVO	Barra de abas e janela de pré-visualização
`js/main.js`	Monta a tela, os menus, os atalhos, o salvamento e o rascunho, e chama o install(app) de cada módulo de features
`js/features/*.js` NOVO	layers.js (menus Camada e Quadro), selection.js (menu Seleção), color.js (menu Cor) e files.js (abrir, abas e exportações), ligados por install(app)

Ponto de extensão: features e app NOVO

Os recursos da 2.1.0 não se espalham pelo `main.js`: cada área é um módulo em `js/features/` que exporta `install(app)`. O `main.js` chama o `install` de `layers.js`, `selection.js`, `color.js` e `files.js` antes de montar as ferramentas, as opções e os menus. O objeto `app` traz `editor`, `viewport`, `controller`, `commands` (cada comando é `{ label, shortcut, fn, checked? }`), `execute(id)`, `addMenuItems(menu, ids, after)`, `addMenu(title, ids)`, `addShortcut({ key, ctrl, shift, alt, run })`, `addToolOptions(builder)`, `rebuildMenus()`, `notify` e a paleta.

```
export function install(app) {
  Object.assign(app.commands, {
    myCommand: { label: 'Meu comando', shortcut: '', fn: () => app.notify('Olá!') },
  });
  app.addMenuItems('Imagem', ['-', 'myCommand']);
  return app;
}
```

Modos de mistura: a fórmula NOVO

O modo normal usa a mesma conta inteira da 2.0, então desenhos antigos dão exatamente os mesmos pixels. Os outros seguem a especificação Compositing and Blending Level 1 do W3C, com valores entre 0 e 1:

```
αs  = (alfa / 255) · opacidade
Cs' = (1 − αb) · Cs + αb · B(Cb, Cs)
αo  = αs + αb · (1 − αs)
Co  = (αs · Cs' + αb · Cb · (1 − αs)) / αo
canal = floor(Co · 255 + 0,5)     alfa = floor(αo · 255 + 0,5)
```

Aqui `B` é a função do modo (por exemplo, `Cb · Cs` no multiplicar), `Cb` a cor de baixo e `Cs` a de cima. A adição é `min(1, b + s)`, a subtração `max(0, b − s)` e a divisão `b / s`, com `s = 0` dando 1. O JavaScript (`core/blend.js`), o Python (`editor/blend.py`) e o Java (`Blend.java`) avaliam a conta na mesma ordem, e um teste de paridade com 30 documentos aleatórios exige os mesmos pixels nos três.

Fluxo de eventos

O `Editor` estende `EventTarget` e avisa as partes da tela com eventos: `pixels` (redesenhar a matriz), `document` (estrutura mudou, refazer listas), `frame`, `layer`, `color`, `state`, `selection`, `history`, `zoom` e `saved`. Nenhum painel conhece os outros; cada um só escuta os eventos de que precisa.

O visor (`Viewport`) nunca redesenha mais de uma vez por quadro de vídeo: os pedidos se acumulam até o próximo `requestAnimationFrame`. A composição só é refeita quando os pixels mudam; mover o mouse, a visão ou as formigas da seleção só redesenha a imagem já composta.

As ferramentas novas seguem o mesmo padrão das antigas: abrem um `Stroke`, pintam e concluem. O mover pixels desfaz visualmente o traço a cada movimento do mouse e repinta na posição nova, e o degradê recalcula a prévia; nos dois casos, o histórico só recebe o resultado final.

Servidor Django

Arquivo	Responsabilidade
`crivo/settings.py`	Configuração por variáveis de ambiente (aceitando os nomes antigos da 1.x como alternativa), SQLite ou PostgreSQL, cabeçalhos de segurança, HSTS, limites e recusa de chave fraca em produção
`editor/models.py`	`Project`, `Layer`, `Frame`, `Cell`, `Tag`, `Slice`, `Tileset`, `Revision`, `Palette` e `PaletteColor`
`editor/services.py`	Validação do `.crivo` (convertendo a versão 1 com `upgrade_document`), gravação em transação, versões, conflitos, composição

Arquivo	Responsabilidade
`editor/api.py`	Rotas JSON, tratamento de erros, exportação e PNG
`editor/rle.py`	RLE e composição em Python, compatíveis com o JavaScript
`editor/blend.py` NOVO	Os 19 modos de mistura e a composição de grupos e camadas de referência, com a mesma conta do editor
`editor/export.py`	JSON, CSV, JavaScript, Python, C e Java pelo servidor, e o codificador de PNG em Python puro
`editor/security.py`	Token de API como alternativa ao CSRF, limite de escritas por IP e cabeçalhos de segurança
`editor/views.py`	Entrega o editor com o `<base>` apontando para os arquivos estáticos

O detalhe das tabelas está no PDF de banco de dados; a migração `0004_aseprite_features` NOVO acrescenta as colunas e as tabelas da 2.1.0 (rode `python manage.py migrate` ao atualizar).

Código em inglês desde a 2.0.0

Na versão 2.0.0, todo o código do Crivo passou para o inglês: identificadores, comentários, nomes de arquivos e pastas (o núcleo do editor agora fica em `js/core` e a ferramenta em Java, em `tools/java`), rotas e campos da API (`/api/v2`), chaves do formato `.crivo`, tabelas e colunas do banco, variáveis de ambiente, comandos da ferramenta em Java e scripts do npm. O que aparece na tela continua em português: menus, diálogos, avisos, as mensagens de erro da API e os textos da linha de comando. O comportamento não mudou.

O que foi feito na 1.x continua abrindo: arquivos `.crivo` da versão 1 são convertidos na leitura, a migração `0003_english_schema` renomeia o banco sem perder dados, o editor migra sozinho os projetos e as preferências guardados no navegador e no app desktop, as variáveis de ambiente antigas ainda são lidas e a ferramenta em Java aceita os comandos antigos como apelidos.

Capítulo 14

API

Todas as rotas ficam em `/api/v2`, falam JSON em UTF-8 e respondem erros como `{"error": "mensagem"}`, com a mensagem em português. `GET /api/v2` mostra o nome, a versão e a lista das rotas. A API da 1.x, com rotas e campos em português, foi removida na 2.0.0: scripts antigos precisam trocar o endereço e os nomes dos campos.

Autenticação

Leituras são livres. Pedidos que alteram dados (`POST`, `PUT` e `DELETE`) seguem a proteção CSRF do Django: o editor recebe o cookie `csrftoken` ao abrir e o devolve no cabeçalho `X-CSRFToken`. Scripts e outros programas podem, em vez disso, mandar `Authorization: Bearer <token>`, desde que `CRIVO_API_TOKEN` esteja definido no servidor. O token é comparado em tempo constante, e um token errado não libera nada. Alterações aceitas pelo CSRF ou pelo token contam para o limite de escritas por IP; pedidos recusados com 403 (por exemplo, forjados por outro site) não gastam a cota de ninguém.

Rotas

Método e rota	Corpo ou parâmetros	Resposta
`GET /health`	—	`{status, version, database}`; 503 se o banco não responder
`GET /projects`	`q` filtra pelo nome	`{projects: [...]}` com id, nome, tamanho, quadros, camadas, revisão e miniatura
`POST /projects`	`data` (.crivo), `thumbnail` (PNG em data URL), `name`	201 com o resumo do projeto
`GET /projects/{id}`	—	Resumo + `data` no formato .crivo (versão 2)
`PUT /projects/{id}`	`data`, `thumbnail`, `revision`, `name`	Resumo com a nova revisão; 409 se `revision` estiver desatualizada, com `current_revision`
`DELETE /projects/{id}`	—	204
`GET /projects/{id}/thumbnail`	—	PNG
`GET /projects/{id}/png`	`frame` (N, padrão 1), `scale` (1 a 32), `download`	PNG do quadro composto; o lado final não passa de 2048 px
`GET /projects/{id}/export`	`format` (json, csv, js, py, c, java), `mode` (indices, hex), `frame` (`all` ou N), `download`	Texto no formato pedido; com `download`, o `.java` sai com o nome da classe (`Pocao.java`)
`GET /projects/{id}/revisions`	—	`{current_revision, revisions: [...]}`

Método e rota	Corpo ou parâmetros	Resposta
`POST /projects/{id}/revisions/{number}/restore`	—	Resumo com a nova revisão
`GET /palettes`	`presets=0` esconde as predefinidas	`{palettes: [...]}`
`POST /palettes`	`name`, `colors` (1 a 256)	201 com a paleta
`GET/PUT/DELETE /palettes/{id}`	`name`, `colors`	Paleta, paleta alterada ou 204; 403 nas predefinidas

O resumo de um projeto traz `id`, `name`, `width`, `height`, `fps`, `revision`, `frames` e `layers` (quantidades), `thumbnail_url`, `created_at` e `updated_at`. Uma paleta traz `id`, `slug`, `name`, `description`, `preset`, `colors` e `updated_at`. O campo `data` também aceita um projeto no formato .crivo antigo, que é convertido antes de gravar.

Campos da 2.1.0 NOVO

O `data` aceita e devolve as chaves opcionais do formato (capítulo 10): `colorMode`, `tags`, `slices` e `tilesets`; nas camadas, `blendMode`, `kind`, `parent`, `collapsed`, `tileset` e `reference`; nas células, `link` e `tiles`. Elas são validadas e guardadas nas tabelas novas; um projeto sem essas chaves é gravado e devolvido exatamente como na 2.0. O PNG e as exportações compõem grupos e modos de mistura e deixam de fora as camadas de referência. Limites: 256 tags, 256 slices e 32 tilesets por projeto, tiles de até 256 × 256 pixels e até 4096 tiles por tileset.

Códigos de resposta

Código	Quando
200 / 201 / 204	Deu certo
400	Parâmetro inválido (formato de exportação desconhecido, quadro ou escala fora do intervalo)
403	Sem CSRF ou token, ou tentativa de alterar uma paleta predefinida
404	Projeto, paleta ou versão não existe
409	Conflito de revisão ao salvar
413	Corpo maior que `CRIVO_MAX_UPLOAD_MB`
422	Conteúdo inválido: JSON quebrado, que não é um objeto ou aninhado demais, tamanho fora de 1 a 1024, ids repetidos, RLE que não fecha com o tamanho, cor inválida, miniatura que não é PNG; na 2.1.0, também modo de mistura desconhecido, grupo que não existe ou contém a si mesmo, tag apontando para um quadro que não existe, vínculo inválido e tiles fora do tileset
429	Alterações demais do mesmo IP no último minuto; o cabeçalho `Retry-After` diz quantos segundos esperar
503	Banco de dados fora do ar

Exemplos

```
TOKEN=seu-token
curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
     -d "{\"data\": $(cat examples/heart.crivo)}" \
     http://localhost:8080/api/v2/projects

curl "http://localhost:8080/api/v2/projects/<id>/export?format=csv&frame=1"
curl -o Pocao.java "http://localhost:8080/api/v2/projects/<id>/export?format=java&frame=all"
curl -o potion.png "http://localhost:8080/api/v2/projects/<id>/png?frame=2&scale=8"

curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
     -d '{"name": "Terra", "colors": ["#5c4034", "#7a6e46", "#e6ddce"]}' \
     http://localhost:8080/api/v2/palettes
```

Capítulo 15

Ferramenta em Java

A pasta `tools/java` tem uma linha de comando que só precisa do JDK 21. Ela tem o próprio leitor de JSON, decodifica o RLE, compõe as camadas com a mesma fórmula do editor e grava PNG com `javax.imageio`. Os scripts `crivo.sh` e `crivo.cmd` compilam as classes na primeira vez, em `tools/java/out`.

Comando	O que faz
`info <projeto>`	Nome, tamanho, quadros, FPS, camadas (com grupos, tipo, opacidade e modo de mistura), modo de cor, quantidade de cores, tags, slices e tilesets
`tags <projeto>` NOVO	As tags: nome, quadros, direção, repetições e cor
`palette <projeto>`	Cores usadas, da mais usada para a menos usada, com a contagem de pixels
`png <projeto> <saida.png> [--frame N] [--scale N]`	Um quadro em PNG, ampliado por vizinho mais próximo
`sheet <projeto> <saida.png> [--layout ...] [--columns N] [--spacing N] [--scale N]`	Sprite sheet horizontal, vertical ou em grade (`horizontal` , `vertical` ou `grid`)
`matrix <projeto> [--frame N|all] [--mode indices|argb] [--class Nome]`	Classe Java com a matriz, impressa na saída padrão; em projetos grandes, no formato compacto que compila (capítulo de importação e exportação)
`check <projeto>`	Confere se o arquivo é um projeto válido

```
cd tools/java
./crivo.sh info ../../examples/potion.crivo
./crivo.sh tags ../../examples/blend-modes.crivo
./crivo.sh sheet ../../examples/heart.crivo heart.png --scale 4 --spacing 2
./crivo.sh matrix ../../examples/game-boy-slime.crivo --class Slime > Slime.java
./test.sh
```

Desde a 2.0.0, os comandos e as opções estão em inglês. Os nomes em português da 1.x continuam aceitos como apelidos, então scripts antigos não quebram; a ajuda e as mensagens seguem em português. A ferramenta lê arquivos `.crivo` nas versões 1 e 2, então todos os projetos, antigos e novos, abrem sem mudança.

Na 2.1.0 NOVO, ela lê as chaves opcionais novas, compõe grupos e os 19 modos de mistura com a mesma conta do editor e do servidor e deixa as camadas de referência fora do PNG e da sprite sheet. Para o exemplo `blend-modes.crivo`, o `tags` mostra:

```
Tags: 1
  - Nascer do sol: quadros 1 a 2, pingue-pongue (#ffa300)
```

Capítulo 16

Configuração

O servidor lê tudo de variáveis de ambiente. Com Docker, elas ficam no arquivo `.env`, criado pelos scripts de início a partir do `.env.example`.

Variável	Padrão	O que faz
`CRIVO_PORT`	`8080`	Porta publicada pelo Docker
`CRIVO_DEBUG`	`true` fora do Docker, `false` no Docker	Modo de desenvolvimento do Django
`CRIVO_SECRET_KEY`	chave de desenvolvimento	Chave do Django. Obrigatória com `CRIVO_DEBUG=false`, com pelo menos 32 caracteres; os scripts geram uma aleatória
`CRIVO_ALLOWED_HOSTS`	`localhost,127.0.0.1`	Nomes pelos quais o servidor é acessado, separados por vírgula
`CRIVO_CSRF_TRUSTED_ORIGINS`	vazio	Origens completas (com `https://`) quando há proxy ou domínio próprio
`CRIVO_HTTPS`	`false`	Liga cookies seguros, HSTS e confia no `X-Forwarded-Proto` do proxy
`CRIVO_HSTS_SECONDS`	`31536000`	Duração do `Strict-Transport-Security` (1 ano) quando `CRIVO_HTTPS=true`
`CRIVO_HSTS_SUBDOMAINS`	`false`	Estende o HSTS aos subdomínios (`includeSubDomains`)
`CRIVO_HTTPS_REDIRECT`	`false`	O próprio Django redireciona HTTP para HTTPS. Deixe desligado quando o proxy já faz isso
`CRIVO_API_TOKEN`	vazio	Token para scripts alterarem dados sem CSRF
`CRIVO_WRITE_LIMIT`	`120`	Alterações (`POST`, `PUT`, `DELETE`) por minuto por IP, contadas depois da checagem de CSRF e em cada processo do Gunicorn; 0 desliga
`CRIVO_TRUSTED_PROXY`	`false`	Atrás de um proxy confiável, usa como IP do cliente o último endereço do `X-Forwarded-For` (o que o proxy acrescentou), em vez do IP do proxy. Só ligue se o proxy estiver na frente de todo acesso
`CRIVO_MAX_REVISIONS`	`20`	Versões anteriores guardadas por projeto
`CRIVO_MAX_UPLOAD_MB`	`24`	Tamanho máximo do corpo dos pedidos
`CRIVO_WORKERS`	`2`	Processos do Gunicorn
`CRIVO_DB_ENGINE`	`sqlite`	`postgresql` para usar PostgreSQL

Variável	Padrão	O que faz
`CRIVO_DB_HOST` / `CRIVO_DB_PORT`	`localhost` / `5432`	Endereço do PostgreSQL
`CRIVO_DB_NAME` / `CRIVO_DB_USER` / `CRIVO_DB_PASSWORD`	`crivo` / `crivo` / —	Banco, usuário e senha
`CRIVO_SQLITE_PATH`	`crivo.sqlite3`	Arquivo do SQLite
`CRIVO_TIMEZONE`	`America/Sao_Paulo`	Fuso horário das datas
`CRIVO_LOG_LEVEL`	`INFO`	Detalhe do registro

Nomes antigos das variáveis

Na 2.0.0, sete variáveis ganharam nomes em inglês. O servidor lê primeiro o nome novo e, se ele não estiver definido, ainda aceita o nome da 1.x; esses nomes antigos estão obsoletos, então troque-os no seu `.env`.

Nome na 1.x (obsoleto)	Nome na 2.0.0
`CRIVO_TAMANHO_MAXIMO_MB`	`CRIVO_MAX_UPLOAD_MB`
`CRIVO_HSTS_SEGUNDOS`	`CRIVO_HSTS_SECONDS`
`CRIVO_HSTS_SUBDOMINIOS`	`CRIVO_HSTS_SUBDOMAINS`
`CRIVO_REDIRECIONAR_HTTPS`	`CRIVO_HTTPS_REDIRECT`
`CRIVO_MAX_REVISOES`	`CRIVO_MAX_REVISIONS`
`CRIVO_LIMITE_ESCRITAS`	`CRIVO_WRITE_LIMIT`
`CRIVO_PROXY_CONFIAVEL`	`CRIVO_TRUSTED_PROXY`

Antes de publicar na internet

- Use uma `CRIVO_SECRET_KEY` longa e aleatória e nunca a coloque no repositório. Com `CRIVO_DEBUG=false`, o Crivo não inicia com a chave de desenvolvimento nem com uma chave curta.
- Coloque o domínio em `CRIVO_ALLOWED_HOSTS` e `https://seu-dominio` em `CRIVO_CSRF_TRUSTED_ORIGINS`.
- Sirva por trás de um proxy com HTTPS e defina `CRIVO_HTTPS=true`, que também liga o HSTS.
- Defina um `CRIVO_API_TOKEN` longo só se algum script precisar alterar dados, e mantenha o `CRIVO_WRITE_LIMIT` ligado.
- O Crivo não tem contas de usuário: qualquer pessoa com acesso ao endereço vê e altera os projetos. Para uso público, proteja o endereço no proxy (autenticação básica, VPN ou rede interna).
- Faça backup do volume do PostgreSQL (veja o PDF de banco de dados), principalmente antes de atualizar da 1.x para a 2.0.0 ou de rodar a migração `0004_aseprite_features` da 2.1.0.

Capítulo 17

Qualidade e segurança

A versão 1.1.0 passou por uma rodada de testes de qualidade e de segurança. Este capítulo mostra o que o servidor faz para se proteger, o que foi verificado e como repetir as verificações.

Cabeçalhos de segurança

Toda resposta do servidor, do editor à API, sai com estes cabeçalhos:

Cabeçalho	Valor e motivo
`Content-Security-Policy`	`default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; font-src 'self'; connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'`. Só scripts do próprio Crivo rodam; um nome de projeto ou camada com código malicioso não executa
`Permissions-Policy`	Câmera, microfone, geolocalização, pagamento, USB, serial, Bluetooth, MIDI e sensores de movimento desligados
`Cross-Origin-Opener-Policy`	`same-origin`: outra aba não consegue manipular a janela do editor
`Cross-Origin-Resource-Policy`	`same-origin`: outros sites não embutem as imagens e dados da API
`X-Content-Type-Options`	`nosniff`: o navegador respeita o tipo declarado (uma miniatura nunca vira script)
`Referrer-Policy`	`same-origin`: endereços de projetos não vazam para outros sites
`X-Frame-Options`	`DENY`: o editor não pode ser embutido em outra página (clickjacking)
`Cache-Control` (API JSON)	`no-store`: dados de projetos não ficam em caches intermediários
`Strict-Transport-Security`	Com `CRIVO_HTTPS=true`: `max-age` de `CRIVO_HSTS_SECONDS`, para o navegador nunca mais usar HTTP; `includeSubDomains` com `CRIVO_HSTS_SUBDOMAINS=true`

Proteções no servidor

Ameaça	Proteção
Falsificação de pedidos (CSRF)	Cookie `SameSite=Strict` e cabeçalho `X-CSRFToken` em toda alteração; scripts usam o token da API
Adivinhar o token	Comparação em tempo constante (`hmac.compare_digest`); token errado é ignorado e o pedido cai no CSRF
Excesso de pedidos	Limite de escritas por IP numa janela de um minuto, com resposta 429 e `Retry-After`. Contado depois do CSRF, para que pedidos forjados não esgotem a cota de ninguém; atrás de proxy, o IP real vem do `X-Forwarded-For` com `CRIVO_TRUSTED_PROXY=true`
Corpo gigante	Corte em `CRIVO_MAX_UPLOAD_MB` com resposta 413

Ameaça	Proteção
JSON aninhado (recursão)	Um JSON com milhares de níveis responde 422 em vez de derrubar o processo com erro 500
Arquivo disfarçado de miniatura	Só aceita PNG conferido pela assinatura dos bytes, até 512 KB
Projetos que esgotam a memória	Limites de lado (1024), camadas (64), quadros (512) e RLE que precisa fechar com o tamanho; PNG do servidor limitado a 2048 px de lado
Nome de arquivo malicioso no download	O nome do projeto vira um identificador só com letras, números e hífen antes de ir para o `Content-Disposition`
Configuração de produção insegura	Com `CRIVO_DEBUG=false`, o servidor não inicia com a chave de desenvolvimento ou com menos de 32 caracteres
Preferências adulteradas no navegador	Valores do `localStorage` fora da lista permitida (uma matriz de Bayer 16, uma ferramenta que não existe, pincel de 999 px) são ignorados e voltam ao padrão
Injeção de SQL	Todo acesso ao banco passa pelo ORM do Django, com parâmetros, sem SQL montado com texto do usuário

No editor, textos vindos do usuário (nomes de projetos, camadas e paletas) são sempre inseridos com `textContent`, nunca com `innerHTML`; o `innerHTML` só recebe os ícones em SVG gerados pelo próprio código.

O `Crivo.html` e o app desktop levam a própria Content-Security-Policy numa tag `<meta>`: só executam o script embutido, conferido por hash SHA-256, e não abrem nenhuma conexão de rede. O `Crivo.exe` roda sem permissão de administrador, não abre portas e desliga as ferramentas de desenvolvedor e o menu de contexto do navegador.

Testes por conjunto

Conjunto	Comando	Testes	O que cobre
Núcleo em JavaScript	`npm test`	107	Tudo o que já era coberto e mais seleção, colagem, aparo, deslocamento, contorno, paleta, filtros de cor, Bayer, degradês, pixel perfect e GIF (LZW de ida e volta passando de 4096 códigos, animação com transparência, atrasos e repetição, e mais de 255 cores); na 2.1.0, modos de mistura, grupos, tags, vínculos, tilemaps, seleções com máscara, ferramentas novas, tinta, pincel, transformação, modos de cor, ajustes, paleta, `.aseprite`, APNG, ZIP e JSON do Aseprite
API Django	`python manage.py test editor`	72	Ciclo de projetos e versões, exportação em seis formatos, PNG do servidor, o conjunto de segurança, as migrações e os campos da 2.1.0

Conjunto	Comando	Testes	O que cobre
Ferramenta Java	`bash tools/java/test.sh`	36	RLE, JSON, composição, identificadores, geração de classe, PNG ampliado e uma classe grande que compila com o compilador do JDK e devolve os mesmos valores; na 2.1.0, chaves opcionais, modos de mistura, grupos, `info` e `tags`
Ponta a ponta	`python -m unittest discover -s tests/e2e -v`	42	Abre o editor no Chromium com Playwright de dois jeitos: pelo servidor Django (20) e pelo `Crivo.html` direto do disco (22). Usa as ferramentas, espelha a seleção, substitui cor, baixa um GIF, desfaz e refaz, salva no servidor e no navegador, reabre o `.crivo`, confere a integração com o app desktop, laço, varinha mágica, polígono, curva e modos de tinta, e que o console não tem erros
Paridade de composição	`npm test`, `manage.py test` e `test.sh`	30 docs	Documentos aleatórios com os 19 modos de mistura, grupos (inclusive "pass-through" normais a 100%) e camadas ocultas e de referência, compostos no JavaScript, no Python e no Java: os três precisam dar os mesmos pixels (`tests/fixtures/blend-parity.json`, gerado por `tests/js/fixtures/make_blend_fixtures.mjs`)
Arquivo único e app desktop	`npm run desktop -- --check`	build	Gera de novo o `Crivo.html` e o `Crivo.exe` e confere que são idênticos aos do repositório, byte a byte

> Na versão 2.1.0, o teste de paridade achou uma diferença rara: o servidor arredondava a opacidade de um jeito diferente do editor quando o valor caía exatamente no meio (como 52,5). Foi corrigido, e o servidor, a ferramenta em Java e o editor geram exatamente os mesmos pixels.

> Na versão 1.1.0, uma revisão de código independente apontou 10 problemas: histórico corrompido ao desfazer durante um arraste, colagem que furava o fundo, degradê pintando com um clique, classe Java que não compilava, nome do `.java` baixado, `int16_t` pequeno demais no C, limite de escritas atrás de proxy, setas agindo fora do menu aberto, preferências inválidas no navegador e `Esc` na seleção. Todos foram corrigidos e cobertos por testes.

O teste ponta a ponta precisa do Playwright e do Chromium, instalados uma vez com `pip install playwright` e `python -m playwright install chromium`.

O conjunto `editor/tests/test_security.py` confere cada item das tabelas acima: presença e valor dos cabeçalhos, CSRF obrigatório sem token, token errado recusado, limite de escritas com 429, JSON aninhado com 422, miniatura falsa com 422, corpo grande com 413, recusa da chave insegura, IDs inválidos com 404 e o nome do arquivo baixado sem caracteres perigosos.

Análise estática

Ferramenta	Comando	Resultado
Ruff (estilo, erros e segurança no Python; regras em `ruff.toml`)	`ruff check .`	Ruff (regras E, F, W, B, S, UP, SIM): 0 problemas
Bandit (segurança no Python)	`bandit -r editor crivo -x editor/tests`	Bandit (editor/ e crivo/): 0 problemas
Checagem de produção do Django	`python manage.py check --deploy` (com `CRIVO_DEBUG=false` e `CRIVO_HTTPS=true`)	manage.py check --deploy com CRIVO_DEBUG=false e CRIVO_HTTPS=true: só 3 avisos opcionais (HSTS em subdomínios, preload e redirecionamento HTTPS, que ficam a cargo do proxy e podem ser ligados por CRIVO_HSTS_SUBDOMAINS e CRIVO_HTTPS_REDIRECT)
go vet (código do app desktop)	`GOOS=windows go vet ./...` em `desktop/`	0 problemas
Busca de `innerHTML` com dados do usuário	revisão do código do editor	Auditoria do editor: nenhum uso de innerHTML com dados do usuário (só ícones SVG gerados no próprio código); nomes de camadas e projetos entram sempre como texto

Integração contínua

O workflow de CI do GitHub Actions roda Ruff e Bandit e depois os conjuntos de testes a cada push e pull request, com a API testada contra um PostgreSQL 17 de verdade, confere se não faltam migrações, gera os PNGs dos exemplos com a ferramenta em Java e constrói a imagem Docker. Também confere se o `Crivo.html` e o `Crivo.exe` estão em dia com o código: o `.exe` é gerado de novo e comparado byte a byte com o do repositório, e o código Go do app passa pelo `go vet`.

Capítulo 18

Personalizando o Crivo

Uma paleta fixa nova

Acrescente um objeto em `PRESET_PALETTES`, no arquivo `editor/static/editor/js/core/palettes.js`. Para a API listar a mesma paleta, repita em `PRESET_PALETTES`, no `editor/palettes.py`, e rode `python manage.py migrate` num banco novo (ou chame `sync_presets` num shell do Django).

```
{
  slug: 'terra',
  name: 'Terra',
  description: 'Tons de solo para cenários.',
  colors: list('3b2a20 5c4034 7a6e46 a89a6a e6ddce'),
},
```

Uma ferramenta nova

Toda ferramenta é registrada na lista `TOOLS` de `js/ui/tools.js` (id, nome, atalho, ícone e opções). O comportamento fica nos métodos `pointerDown`, `pointerMove` e `pointerUp` do `Controller`. O padrão é sempre o mesmo: abrir um traço, pintar e concluir, para o histórico funcionar sozinho. Para respeitar a seleção, pinte por meio do método que confere se o pixel está dentro dela, como fazem o lápis e o pontilhado.

```
const stroke = this.editor.newStroke();
if (!stroke) return;
stroke.paint(y * width + x, color);
this.editor.emit('pixels');
this.editor.finishStroke(stroke, 'Minha ferramenta');
```

Um recurso com menu próprio NOVO

Para algo maior que uma ferramenta, crie um módulo em `js/features/` com `install(app)` (capítulo 13), registre comandos em `app.commands`, ponha-os num menu com `app.addMenuItems` ou `app.addMenu`, crie atalhos com `app.addShortcut` (sem repetir uma combinação que já existe) e chame o `install` no `main.js`, junto dos outros. O gerador do `Crivo.html` só aceita `import { a, b } from './x.js'` e `export function|class|const`.

Um modo de mistura novo

Acrescente o nome em `BLEND_MODES` e o rótulo em `BLEND_LABELS` (`js/core/blend.js`), a mesma fórmula em `editor/blend.py` e em `Blend.java`, e o nome em `BLEND_MODES` do `editor/models.py` (com uma migração para a restrição `layer_blend_mode_valid`). Gere de novo o arquivo do teste de paridade com `tests/js/fixtures/make_blend_fixtures.mjs`.

Um filtro novo

Escreva uma função pura em `js/core/filters.js` que recebe os pixels da célula, a largura e a altura e devolve os pixels novos, teste no Node e registre um comando em `COMMANDS` e no menu **Imagem** de `js/main.js`. Os filtros existentes mostram como aplicar o resultado num traço, para que o filtro respeite a seleção e possa ser desfeito.

Um formato de exportação novo

Escreva uma função que recebe o resultado de `buildMatrices` (largura, altura, modo, paleta e quadros) e devolve texto, registre-a em `DATA_FORMATS` e em `generateData`, no arquivo `js/core/export.js`. O diálogo de exportação passa a oferecer o formato automaticamente. Para a API oferecer o mesmo formato, acrescente-o em `FORMATS` e em `generate`, no `editor/export.py`.

Ícones, cores e atalhos

Os ícones são bitmaps em `js/ui/icons.js`: cada linha é uma string em que `#` é um pixel aceso. As cores da interface são variáveis CSS no início de `css/crivo.css`, separadas para o tema claro e o escuro. Os atalhos ficam na função `shortcuts` de `js/main.js`.

Capítulo 19

Atalhos do teclado

Tecla	Ação
`B`, `E`, `F`, `L`, `R`, `O`, `I`, `H`	Lápis, borracha, balde, linha, retângulo, elipse, conta-gotas e mover a visão
`D`, `U`, `Q`, `S`, `V`	Pontilhado, degradê pontilhado, clarear e escurecer, seleção retangular e mover pixels
`Alt` + clique	Pegar a cor com qualquer ferramenta de desenho
Clique direito	Desenhar com a cor secundária (escurecer, no clarear e escurecer)
`Shift` (lápis)	Reta a partir do último ponto
`Shift` (formas e seleção)	45°, quadrado ou círculo perfeito
`Alt` (formas)	Desenhar a partir do centro
`Esc`	Cancelar o traço, a forma ou o arraste em andamento (na seleção, volta a anterior); com um menu aberto, fecha o menu
Botão do mouse pressionado	Os outros atalhos ficam desligados até soltar o botão
Setas / `Shift` + setas	Com Mover pixels: mover a seleção 1 ou 8 pixels. Com um menu aberto, as setas só andam pelo menu
`X`	Trocar cor principal e secundária
`[` e `]`	Diminuir e aumentar o pincel (1 a 32)
`M`	Alternar a simetria
`G`	Grade
`K`	Papel cebola
`T`	Modo ladrilho
`Shift` + `T`	Deslocar meia matriz (ladrilho)
`Shift` + `H` / `Shift` + `V`	Espelhar na horizontal / na vertical (seleção ou camada)
`Espaço` + arrastar	Mover a visão
Roda do mouse	Zoom no ponto do cursor
`+` e `−`	Aproximar e afastar
`0` e `1`	Ajustar à tela e tamanho real
`,` e `.`	Quadro anterior e próximo

Tecla	Ação
`Enter`	Tocar ou pausar a animação
`Shift` + `B`, `Shift` + `L`, `Shift` + `D`, `Shift` + `O` NOVO	Spray, curva, contorno livre e polígono
`Shift` + `Q`, `Shift` + `J` NOVO	Desfocar e embaralhar
`Shift` + `S`, `A`, `Shift` + `A`, `W` NOVO	Seleção elíptica, laço, laço poligonal e varinha mágica
`Shift` / `Alt` / `Shift` + `Alt` (seleção) NOVO	Somar, subtrair ou fazer a interseção com a seleção atual
`Enter`, duplo clique, `Backspace` e `Esc` NOVO	No polígono, no laço poligonal e na curva: fechar, tirar o último ponto e cancelar
`Delete`	Apagar a seleção ou, sem seleção, limpar a camada no quadro atual
`Ctrl` + `A` / `Ctrl` + `D`	Selecionar tudo / desmarcar
`Ctrl` + `Shift` + `D` / `Ctrl` + `Shift` + `I` NOVO	Reselecionar / inverter a seleção
`Ctrl` + `B` NOVO	Usar a seleção como pincel
`Shift` + `N` / `Alt` + `N` NOVO	Nova camada / novo quadro
`F2` NOVO	Propriedades da camada
`Ctrl` + `U` / `Ctrl` + `M` NOVO	Matiz, saturação e luminosidade / curva de cores
`Ctrl` + `C` / `X` / `V`	Copiar, recortar e colar
`Ctrl` + `Z`	Desfazer
`Ctrl` + `Y` ou `Ctrl` + `Shift` + `Z`	Refazer
`Ctrl` + `S`	Salvar
`Ctrl` + `O`	Abrir arquivo (`.crivo` , `.aseprite` ou `.ase`) numa aba nova
`Ctrl` + `Tab` / `Ctrl` + `Shift` + `Tab` NOVO	Próxima aba / aba anterior
`F7` NOVO	Mostrar ou ocultar a pré-visualização
`Ctrl` + `E`	Exportar imagem
`Ctrl` + `Shift` + `E`	Exportar dados da matriz
`Ctrl` + `Shift` + `G`	Exportar GIF animado
`Ctrl` + `Alt` + `C`	Tamanho da matriz

Tecla	Ação
`?`	Lista de atalhos
`F11` (só no app desktop)	Tela cheia e volta para a janela maximizada
`Alt` + `F4` (só no app desktop)	Fechar o Crivo, gravando o rascunho antes

Capítulo 20

Solução de problemas

Problema	O que fazer
A barra de status mostra **Modo local**, mas o servidor está ligado	Abra o editor pelo endereço do servidor (http://localhost:8080 ou :8000), não pelo `npm run serve` nem pelo `Crivo.html`. Confira http://localhost:8080/api/v2/health
Tela em branco ao abrir o `index.html` direto do disco	Navegadores bloqueiam módulos por `file://`. Abra o `Crivo.html`, feito para abrir direto do disco, ou use `npm run serve` ou o servidor Django
Aparece **O Windows protegeu o computador** ao abrir o `Crivo.exe`	Clique em **Mais informações** e depois em **Executar assim mesmo**. O aviso aparece porque o executável não tem assinatura digital paga
O `Crivo.exe` avisa que falta o WebView2	Aceite abrir a página da Microsoft, instale o Microsoft Edge WebView2 Runtime e abra o Crivo de novo
Pesquisei **Crivo** no menu Iniciar e não apareceu	Abra o `Crivo.exe` uma vez com dois cliques: na primeira abertura ele cria o atalho no menu Iniciar. A pesquisa do Windows pode levar alguns segundos para encontrá-lo
Quero sair da tela cheia no app desktop	`F11` alterna a tela cheia; `Alt` + `F4` ou **Arquivo → Sair do Crivo** fecha o app, gravando o rascunho antes
Os projetos do `Crivo.html` sumiram	Eles ficam no navegador em que o arquivo foi aberto; em outro navegador ou numa janela anônima a lista começa vazia. Guarde cópias com **Arquivo → Baixar arquivo .crivo**
Não consigo desenhar	A camada selecionada está oculta ou bloqueada; o aviso no canto diz qual. Confira também se a cor principal não está com alfa 0
Só uma parte do desenho aceita tinta	Há uma seleção ativa (o contorno tracejado). Desmarque com `Ctrl` + `D`
O balde pintou a imagem toda	Desligue **Só a área contígua**, diminua a tolerância, troque **Ler de** para a camada atual ou faça uma seleção antes
O clarear e escurecer não muda a cor	No modo rampa, a cor já é a mais clara (ou escura) da paleta atual. Escolha outra paleta, acrescente tons com o gerador ou use o modo luminosidade
Salvar dá conflito	O projeto foi salvo em outra aba depois que você o abriu. Salve por cima ou abra a versão mais nova; nada se perde, as versões ficam guardadas
Um script que usava a API parou de funcionar na 2.0.0	A API passou para `/api/v2`, com rotas, parâmetros e campos em inglês (capítulo 14). Troque o endereço e os nomes dos campos; o formato `.crivo` antigo continua aceito no campo `data`
Depois de atualizar para a 2.1.0, o servidor reclama de colunas ou das tabelas `tag`, `slice` ou `tileset`	Falta aplicar a migração `0004_aseprite_features`: rode `python manage.py migrate` (com Docker, `docker compose up -d --build`). Ela só acrescenta colunas e tabelas; nada é reescrito
Um `.aseprite` não abre	O Crivo abre arquivos de 32, 16 (tons de cinza) e 8 bits (indexado) até 1024 × 1024. O aviso diz o que faltou; abra no Aseprite e salve de novo, ou exporte em PNG e use **Importar imagem**

Problema	O que fazer
Uma camada some na exportação	Ela é uma camada de referência (selo **Ref**), que nunca é exportada. Desligue **Camada de referência** em **Propriedades da camada...** (`F2`) se quiser exportá-la
A animação toca só uma parte dos quadros	O quadro atual está dentro de uma tag, e a reprodução fica presa nela. Vá para um quadro fora da tag ou exclua a tag em **Quadro → Excluir a tag do quadro**
Pintei um quadro e outro mudou junto	As células estão vinculadas (o ∞ na linha do tempo). Use **Quadro → Desvincular célula** para fazer uma cópia independente
As cores mudam sozinhas ao desenhar	O projeto está no modo indexado (cada cor vai para a mais próxima da paleta) ou em tons de cinza. Volte para **Cor → Modo de cor: RGBA**
Depois de atualizar, o servidor reclama de tabelas que não existem	Falta aplicar a migração `0003_english_schema`: rode `python manage.py migrate` (com Docker, `docker compose up -d --build`). Os dados não se perdem
Erro 403 ao chamar a API por script	Defina `CRIVO_API_TOKEN` no `.env`, reinicie com `docker compose up -d` e mande `Authorization: Bearer <token>`
Erro 429 ao salvar ou chamar a API	Alterações demais do mesmo endereço no último minuto. Espere os segundos de `Retry-After`, ou aumente `CRIVO_WRITE_LIMIT` para scripts de importação em massa
O servidor não inicia e reclama da `CRIVO_SECRET_KEY`	Com `CRIVO_DEBUG=false`, a chave precisa ser sua e ter pelo menos 32 caracteres. Os scripts de início geram uma; gere outra com `python -c "import secrets; print(secrets.token_urlsafe(50))"`
Erro 400 Bad Request ao acessar por outro endereço	Inclua o nome ou IP em `CRIVO_ALLOWED_HOSTS`
Imagem importada ficou enorme	Aumente o tamanho do pixel no diálogo de importação; o Crivo sugere o valor detectado
`start.cmd` fecha e não abre o navegador	Abra o Docker Desktop, espere ele ficar pronto e rode de novo. `docker compose logs -f web` mostra o que aconteceu
A porta 8080 já está em uso	Troque `CRIVO_PORT` no `.env` e rode `docker compose up -d` de novo
Quero começar do zero	`docker compose down -v` apaga o banco. No modo local, limpe os dados do site no navegador. No app desktop, feche o Crivo e apague a pasta `%LOCALAPPDATA%\Crivo`